

Contents

PCEP™ – Certified Entry-Level Python Programmer (Exam PCEP-30-02)	2
EXAM SYLLABUS PCEP-30-02	2
Section 1: Computer Programming and Python Fundamentals (18%)	3
PCEP-30-02 1.1 – Understand fundamental terms and definitions	3
Topics	3
Question and Answers	3
PCEP-30-02 1.2 – Understand Python’s logic and structure	7
Topics	7
Question and Answers	7
PCEP-30-02 1.3 – Introduce literals and variables into code and use different numeral systems	7
Topics	7
Question and Answers	8
PCEP-30-02 1.4 – Choose operators and data types adequate to the problem	11
Topics	11
Question and Answers	12
PCEP-30-02 1.5 – Perform Input/Output console operations	24
Topics	24
Question and Answers	24
Section 2: Control Flow – Conditional Blocks and Loops (29%)	25
PCEP-30-02 2.1 – Make decisions and branch the flow with the if instruction	25
Topics	25
Question and Answers	25
PCEP-30-02 2.2 – Perform different types of iterations	31
Topics	31
Question and Answers	31
Section 3: Data Collections – Tuples, Dictionaries, Lists, and Strings (25%)	41
PCEP-30-02 3.1 – Collect and process data using lists	41
Topics	41
Question and Answers	41
PCEP-30-02 3.2 – Collect and process data using tuples	55
Topics	55
Question and Answers	55

PCEP-30-02 3.3 Collect and process data using dictionaries	60
Topics.....	60
Question and Answers	60
PCEP-30-02 3.4 Operate with strings	66
Topics.....	66
Question and Answers	66
Sets	68
Question and Answers	68
Section 4: Functions and Exceptions (28%)	68
PCEP-30-02 4.1 – Decompose the code using functions.....	69
Topics.....	69
Question and Answers	69
PCEP-30-02 4.2 – Organize interaction between the function and its environment	75
Topics.....	75
Question and Answers	75
PCEP-30-02 4.3 – Python Built-In Exceptions Hierarchy	80
Topics.....	80
Question and Answers	81
PCEP-30-02 4.4 – Basics of Python Exception Handling.....	86
Topics.....	86
Question and Answers	86

PCEP™ – Certified Entry-Level Python Programmer (Exam PCEP-30-02)

EXAM SYLLABUS PCEP-30-02

Exam Status: Live & Active

The exam consists of four sections:

- Section 1 → 7 items Max Raw Score: 180 (18%)
- Section 2 → 8 items Max Raw Score: 290 (29%)
- Section 3 → 7 items Max Raw Score: 250 (25%)
- Section 4 → 8 items Max Raw Score: 280 (28%)

Last updated: February 23, 2022

Aligned with Exam PCEP-30-02

Section 1: Computer Programming and Python Fundamentals (18%)

Objectives covered by the block (7 exam items)

PCEP-30-02 1.1 – Understand fundamental terms and definitions

Topics

- interpreting and the interpreter, compilation and the compiler
- lexis, syntax, and semantics

Question and Answers

What do you call a computer program which directly executes instructions written in a programming language?

- a) An interpreter
- b) A translator
- c) A compiler

Answer: a) An interpreter

Topic: interpreter

Domain: Fundamentals

Explanation: An interpreter is a computer program that directly executes instructions written in a programming language. It reads the source code line by line, translates it into machine code, and executes it immediately. This process allows for real-time execution of the program.

What is true about compilation?

(Select two answers)

- a) The code is converted directly into machine code executable by the processor
- b) It tends to be faster than interpretation
- c) It tends to be slower than interpretation
- d) Both you and the end user must have the compiler to run your code

Answer: a) and b)

- a) The code is converted directly into machine code executable by the processor
- b) It tends to be faster than interpretation

Topics: compilation interpretation machine code

Domain: Fundamentals

Explanation:

Compilation means, that the source program is translated once (however, this act must be repeated each time you modify the source code) by getting a file containing the machine code.

The execution of the compiled code is usually faster.

Which of the following are true about Python?

- a) Python is a high-level programming language
- b) Python is an interpreted language
- c) Python is a low-level programming language
- d) Python is a compiled language

Answer:

- a) Python is a high-level programming language

b) Python is an interpreted language

Overall explanation

Python is interpreted (because there is no compilation process involved when you run the applications) and is a high-level language like Java or C#. The language C could be considered a low (or lower) level language in turn.

What is the expected output of the following code?

```
print(ord('c') - ord('a'))
```

- a) 3
- b) 2
- c) 1
- d) 0

Answer: b) 2

Topic: ord()

Domain: Fundamentals

Explanation:

ord() returns an integer representing the Unicode character. You do not need to remember the number of each character, but like in the alphabet c is two after a

What is the expected output of the following code?

```
print(chr(ord('z') - 2))
```

- a) a
- b) x
- c) y
- d) z

Answer: b) x

Topics: chr() ord()

Domain: Fundamentals

Explanation:

ord() returns an integer representing the Unicode character.

chr() turns that integer back to the Unicode character.

You don't need to remember the number of every character, but like in the alphabet x is two before z

You have the following file.

index.py:

```
from sys import argv
print(argv[0])
```

You run the file by executing the following command in the terminal.

```
python index.py Hello
```

What is the expected output?

- a) index.py
- b) ImportError
- c) IndexError

d) Hello

Answer: a) index.py

Topic: sys.argv

Domain: Fundamentals

Explanation:

You have to open the folder where you created the index.py file in a terminal and run the file by executing `python index.py Hello`

The first index (the index 0) of the argv variable is always the filename.

If you want Hello to be the output, you have to write `print(argv[1])`

The folder created by Python used to store pyc files is named:

- a) `__cache__`
- b) `__pycfiles__`
- c) `__pycache__`
- d) `__pyc__`

Answer: c) `__pycache__`

Topic: `__pycache__`

Domain: Fundamentals

Explanation:

First please check, if the files index.py and functions.py are created as expected.

Now run the file index.py and the folder `__pycache__` should be created.

In case your Editor/IDE does not show the folder, check in your file system.

Inside the folder should be a file like functions.cpython-39.pyc

You can always delete the folder `__pycache__`

It will be created again the next time you run index.py

Fun Fact:

If you delete the `__pycache__` folder, no problem — Python will recreate it automatically the next time the script runs!

The pyc file contains ...

- a) Python source code.
- b) a Python compiler.
- c) compiled Python bytecode.
- d) a Python interpreter.

Answer: compiled Python bytecode

Topic: pyc

Domain: Fundamentals

Explanation:

First please check, if the files index.py and functions.py are created as expected.

Now run the file index.py and the folder `__pycache__` should be created.

In case your Editor/IDE does not show the folder, check in your file system.

Inside the folder should be a file like functions.cpython-<versionnumber>.pyc

This file contains compiled Python bytecode.

Next time you run index.py Python will use the compiled bytecode for faster execution.

The compiled Python bytecode is stored in files having names ending with:

- a) py
- b) pc

- c) pyc
- d) pyb

Answer: c) pyc

Topic: pyc

Domain: Fundamentals

Explanation:

First please check, if the files index.py and functions.py are created as expected.

Now run the file index.py and the folder __pycache__ should be created.

In case your Editor/IDE does not show the folder, check in your file system.

Inside the folder should be a file like functions.cpython-39.pyc

This file contains compiled Python bytecode.

Next time you run index.py Python will use the compiled bytecode for faster execution.

UTF-8 is ...

- a) a Python version name.
- b) the 9th version of the UTF Standard.
- c) an encoding form of the Unicode Standard.
- d) a synonym for "byte".

Answer: c) an encoding form of the Unicode Standard

Topic: UTF-8

Domain: Fundamentals

Explanation:

UTF-8 is one of three encoding forms of the Unicode Standard.

The others are UTF-16 and UTF-32.

UTF-8 is the most popular one, because it is most flexibel.

UTF-8 requires 8, 16, 24 or 32 bits (one to four bytes)

to encode a Unicode character,

UTF-16 requires either 16 or 32 bits to encode a character,

and UTF-32 always requires 32 bits to encode a character.

ASCII is:

- a) short for American Standard Code for Information Interchange
- b) a standard Python module name
- c) a predefined Python variable name
- d) a character name

Answer: a) short for American Standard Code for Information Interchange

Topic: ASCII

Domain: Fundamentals

Explanation

ASCII stands for the American Standard Code for Information Interchange, which is a character encoding standard for electronic communication. It represents text in computers, telecommunications equipment, and other devices.

PCEP-30-02 1.2 – Understand Python's logic and structure

Topics

- keywords
- instructions
- indentation
- comments

Question and Answers

The following code reads two numbers.

Which of the following is the correct input for the code?

```
x, y = eval(input('Enter two numbers: '))
print(x)
print(y)
```

- a) 3 4
- b) <pre>3 4</pre>
- c) 3, 4
- d) None of the above.

Answer: c) 3, 4

Topics: eval() input() multiple assignment

Domain: Fundamentals

Explanation:

eval() runs the Python code which is passed as an argument.

For multiple assignment you need a comma separated list.

Therefore '3, 4' is the only possible input.

Try it yourself:

```
# x, y = eval(input('Enter two numbers: '))
x, y = eval('3, 4')          # works
# x, y = eval('3 4')         # SyntaxError: ...
# x, y = eval('<pre>3 4</pre>') # SyntaxError: ...
print(x) # 3
print(y) # 4
```

PCEP-30-02 1.3 – Introduce literals and variables into code and use different numeral systems

Topics

- Boolean, integers, floating-point numbers
- scientific notation
- strings
- variables
- naming conventions
- binary, octal, decimal, and hexadecimal numeral systems
- implementing PEP-8 recommendations

Question and Answers

Consider the following python code:

```
x1 = '23'  
y1 = 7  
z1 = x1 * y1
```

```
x2 = 42  
y2 = 7  
z2 = x2 / y2
```

```
x3 = 4.7  
y3 = 1  
z3 = x3 / y3
```

What are the data types of the variables z1, z2 and z3?

- a) str, float, float
- b) str, int, int
- c) str, int, float
- d) str, str, str

Answer: a) str, float, float

Topics: string, integer, float, division operator, multiply operator string concatenation

Domain: Fundamentals

Explanation:

First the string 23 will be multiplied by 7

Second the integer 42 is divided by the integer 7

The result is the float 6.0

Remember the division operator always returns a float

Third the float 4.7 is divided by the integer 1

and surely the result is the float 4.7

What is the expected output of the following code?

```
print(type(1 / 2))
```

- a) <type 'number'>
- b) <type 'tuple'>
- c) <type 'float'>
- d) <type 'double'>
- e) <type 'int'>

Answer: c) <type 'float'>

Topics: division operator type()

Domain: Fundamentals

Explanation: The division operator always returns a float.

What is the expected output of the following code?

```
print(type(1j))
```

- a) <type 'unicode'>
- b) <type 'float'>
- c) <type 'dict'>
- d) <type 'complex'>

Answer: d) <type 'complex'>

Topics: type() complex

Domain: Fundamentals

Explanation:

You can use J or j for complex numbers.

1j is a complex number where the real part is 0 and the imaginary part is 1

What is the expected output of the following code?

```
print(type(+1E10))
print(type(5.0))
print(type('True'))
print(type(False))
```

a) <class 'int'>
 <class 'float'>
 <class 'bool'>
 <class 'bool'>

b) <class 'float'>
 <class 'float'>
 <class 'bool'>
 <class 'bool'>

c) <class 'float'>
 <class 'float'>
 <class 'str'>
 <class 'bool'>

d) <class 'int'>
 <class 'float'>
 <class 'str'>
 <class 'bool'>

Answer: c) <class 'float'>
 <class 'float'>
 <class 'str'>
 <class 'bool'>

Topics: type() float integer string boolean, scientific notation

Domain: Fundamentals

Explanation:

The scientific notation always returns a float

A string with the word 'True' is still a string

Consider the following Python code:

```
distance = 1876.23
amount = +42E7
country = 'Italy'
```

What are the types of the variables distance, amount and country?

- a) double, str, float
b) float, int, str
c) float, str, str
d) float, float, str

Answer: d) float, float, str

Topics: integer float string scientific notation

Domain: Fundamentals

Explanation:

There is no double in Python.

The scientific notation always returns a float

The letter E is called exponent and it does not matter whether you use e or E

What is the data type of x, y, z after executing the following snippet?

```
x = 23 + 42
```

```
y = '23' + '42'
```

```
z = '23' * 7
```

- a) x is int,
- b) y and z are invalid declarations
- c) int, str, int
- d) int, int, int
- e) int, str, str

Answer: 2) int, str, str

Topics: addition operator plus operator string concatenation, multiply operator string concatenation

Domain: Fundamentals

Explanation:

The first one is a normal addition.

The second one is a string concatenation by addition.

The third one is a string concatenation by multiplication.

What is the expected output of the following code?

```
print(float('1.3'))
```

- a) The code is erroneous.
- b) 1,3
- c) 1.3
- d) 13

Answer: 1.3

Topic: float()

Domain: Fundamentals

Explanation:

The function float() does its normal job here and converts the string to a float.

What will be the output after the following? `print(0o1 + 0b1 + 0x1 + 1)`

- a) Python will show an error
- b) 0
- c) 1.3
- d) 4

Answer: d) 4

Overall explanation

0o1 is octal number 1, 0b1 is binary number 1 and 0x1 is hexadecimal number 1. All three of these are equal to 1 in the decimal system, so $0o1 + 0b1 + 0x1 + 1 = 1 + 1 + 1 + 1 = 4$.

What is the expected output of the following code?

```
x, y, z = 3, 2, 1
```

```
z, y, x = x, y, z
```

```
print(x, y, z)
```

- a) 1 2 2
- b) 1 2 3
- c) 3 2 1
- d) 2 1 3

Answer: b) 1 2 3

Topic: multiple assignments

Domain: Fundamentals

Explanation:

Multiple assignment

y stays the same.

x and z change their values.

What will be the output after the following code snippet?

```
age = 30
new_age = age
new_age -= 2
age += 2
print(age, new_age)
a) 32 32
b) 32 28
c) 28 32
d) 30 30
```

Answer: b) 32 28

Overall explanation

Initially, age = 30 and new_age = 30. Then, we subtract 2 from new_age and add 2 to age. This gives us age=32 and new_age=28 in the end.

What will be the output of the following code?

```
if 5 == 5.0:
    print('yes')
else:
    print('no')
a) yes
b) Nothing will be printed to the output
c) Python will show an error
d) No
```

Answer: a) yes

Overall explanation

Python will automatically cast the integer 5 to a double 5.0 to compare the two values.

PCEP-30-02 1.4 – Choose operators and data types adequate to the problem

Topics

- numeric operators: `** * / % // + -`
- string operators: `* +`
- assignment and shortcut operators
- unary and binary operators
- priorities and binding
- bitwise operators: `~ & ^ | << >>`
- Boolean operators: `not, and, or`
- Boolean expressions
- relational operators (`== != > >= < <=`)
- the accuracy of floating-point numbers
- type casting 2

Question and Answers

What is the expected output of the following code?

```
print('Peter' 'Wellert')
```

- a) The code is erroneous.
- b) Wellert
- c) PeterWellert
- d) Peter

Answer: c) PeterWellert

Topic: string concatenation

Domain: Fundamentals

Explanation:

String literals that are delimited by whitespace are automatically concatenated.

What is the expected output of the following code if the user enters 4 and 7?

```
x = input()
y = int(input())
print(x * y)
```

- a) 4444444
- b) 28
- c) 7777
- d) 47

Answer: a) 4444444

Topics: input() int() multiply operator string concatenation

Domain: Fundamentals

Explanation:

Like always input() returns a string

The second one will be cast to an integer

Then the string '4' will be multiplied seven times

and the result is the string '4444444'

Consider the following code.

```
for n in range(1, 6, 1):
    print(??? * 5)
```

What would you insert instead of ???

so that the program prints the following pattern to the monitor?

```
11111
22222
33333
44444
55555
```

- a) 1
- b) -1
- c) 2
- d) n
- e) str(n)

Answer: e) str(n)

Topics: for range() str() multiply operator string concatenation

Domain: Control Flow

Explanation:

range(1, 6, 1) delivers the right numbers 1, 2, 3, 4, 5.

(It would also work without step 1, because that's the default value.)

You need the str() function here to get more numbers.

Otherwise a calculation takes place

and you end up with one number per row.

By string concatenation you get the right result:

'1' * 5 -> '11111'

'2' * 5 -> '22222'

'3' * 5 -> '33333'

'4' * 5 -> '44444'

'5' * 5 -> '55555'

What is the expected output of the following code?

```
def func(message, num=1):  
    print(message * num)
```

```
func('Hello')
```

```
func('Welcome', 3)
```

- a) Hello
WelcomeWelcomeWelcome
- b) Hello
Welcome,Welcome,Welcome
- c) Hello
Welcome Welcome Welcome
- d) Hello
- e) Hello
Viewers

Answer: a) Hello

WelcomeWelcomeWelcome

Topics: def default parameter multiply operator string concatenation

Domain: Functions and Exceptions

Explanation:

In the function a string concatenation by multiplication takes place.

Once with the default value of num (1)

and once with the one passed by argument (3)

Consider the following code.

```
x = 1
```

```
x = x == x
```

The value eventually assigned to x is equal to:

- a) 1
- b) False
- c) 0
- d) True

Answer: d) True

Topic: equal to operator

Domain: Fundamentals

Overall explanation

Try it yourself:

```
x = 1
x = x == x
print(x) # True
x = 1
x = x == x
x = (1 == 1)
x = True
```

Explanation:

x has the value 1

x gets compared with the equal to operator with itself.

It is true, that x is equal to x

True gets stored in the same variable, in x

What value will be assigned to the x variable?

```
z = 3
y = 7
x = y == z and y > z or z > y and z != y
```

- a) True
- b) 0
- c) 1
- d) False

Topics: logical operators relational operators operator precedence

Domain: Fundamentals

Explanation:

The operators here are from three different groups.

"Comparisons, Identity, Membership operators", "Logical AND", "Logical OR".

The three different comparison operators greater than operator

equal to operator not equal to operator have the highest precedence.

Then the logical and operator has a higher precedence

than the logical or operator

Try it yourself:

```
z = 3
y = 7
x = y == z and y > z or z > y and z != y # False
print(x) # False
print(7 == 3 and 7 > 3 or 3 > 7 and 3 != 7) # False
print(False and True or False and True) # False
print((False and True) or (False and True)) # False
print(False or False) # False
print(False) # False
```

What will be the output after this code snippet?

```
print(5 - 3 ** 4 / 4)
```

- a) 2
- b) -15.25
- c) 1.75
- d) -1.75

Answer; b) -15.25

Overall explanation

Exponentiation has the highest priority, so we start with $3 ** 4$, which is $3 * 3 * 3 * 3 = 81$. This leaves us with $5 - 81 / 4$. Division has higher priority than subtraction, so $81 / 4 = 20.25$. Finally, $5 - 20.25 = -15.25$.

What will be the output of the following code?

```
x = 4
y = 5
z = 3
print(x == y or z)
a) 3
b) 0
c) True
d) False
```

Answer: a) 3

This is a tricky question. First, Python will check whether $x == y$. These variables don't have the same value, so the result of this check is False. This means that Python will ignore this part of the expression and will simply be left with $or z$. Because $z=3$, Python will simply print the value to the output.

What will be the output of the following code?

```
print(1 | 0 ^ 1 & ~0)
a) 0
b) True
c) Python will show an error
d) 1
```

Answer: d) 1

Overall explanation

Priorities from highest to lowest: $\sim$ (bitwise not), $\&$ (bitwise and), $\wedge$ (bitwise xor), $|$ (bitwise or).

Bitwise or and bitwise xor are similar, but there is a difference when both bits are 1:

$1 \wedge 1 = 0$

$1 | 1 = 1$

This gives us: $1 | 0 \wedge 1 \& \sim 0 = 1 | (0 \wedge (1 \& (\sim 0))) = 1 | (0 \wedge (1 \& 1)) = 1 | (0 \wedge 1) = 1 | 1 = 1$

What is the expected output of the following code?

```
a = 1
b = 0
c = a & b
d = a | b
e = a ^ b
print(c + d + e)
a) 1
b) 0
c) 3
d) 2
```

Answer: d) 2

Topics: bitwise operators (and, or, xor)

Domain: Fundamentals

Explanation:

1 and 0 is 0

1 or 0 is 1

1 xor 0 is 1

The total is 2

What would you insert instead of ???

so that the program prints True to the monitor?

```
x = 'Peter'
y = 'Peter'
res = ???
print(res)
```

- a) x is y
- b) x < y
- c) x != y
- d) x is not y

Answer: a) x is y

Topics: identity operator less than operator, not equal to operator not

Domain: Fundamentals

Explanation:

A string is an immutable data type.

If you create a second string of the same value,

Python will create a reference to the same object.

Therefore those two strings will have the same identity.

How much will the delivery cost be,

if the order value is 1700 and the state is FL (Florida)?

```
order = int(input('Please enter the order value: '))
state = input('Please enter the state (as postal abbreviation): ')
delivery = 0
```

```
if state in ['NC', 'SC', 'VA']:
    if order <= 1000:
        delivery = 70
    elif 1000 < order < 2000:
        delivery = 80
    else:
        delivery = 90
else:
    delivery = 50
if state in ['GA', 'WV', 'FL']:
    if order > 1000:
        delivery += 30
    if order < 2000 and state in ['WV', 'FL']:
        delivery += 40
    else:
        delivery += 25
print(delivery)
```

- a) 105
- b) 80
- c) 90
- d) 120

Answer: d) 120

Topics: if elif else list membership Operator less than operator, greater than operator less than or equal to operator, add and assign operator chained comparison

Domain: Control Flow

Explanation:

FL is not in ['NC', 'SC', 'VA']

Therefore the else clause executes and the delivery costs start at 50

FL is in ['GA', 'WV', 'FL']

The order value is greater than 1000

That adds another 30 delivery costs.

The order value is less than 2000 and FL is in ['WV', 'FL']

That adds another 40 to the delivery costs.

50 + 30 + 40 -> 120

Which of the code snippets below will print the following to the monitor?

Paul

Mary

Jane

- a)

```
data = ['Peter', 'Paul', 'Mary', 'Jane']
da = data[1:]
for d in data:
    print(d)
```
- b)

```
data = ['Peter', 'Paul', 'Mary', 'Jane']
for d in data:
    if len(d) != 4:
        print(d)
```
- c)

```
data = ['Peter', 'Paul', 'Mary', 'Jane']
for d in data:
    if len(d) == 4:
        print(d)
```
- d)

```
data = ['Peter', 'Paul', 'Mary', 'Jane']
for d in data:
    print(d)
```

Answer: c)

```
data = ['Peter', 'Paul', 'Mary', 'Jane']
for d in data:
    if len(d) == 4:
        print(d)
```

Topics: list for if len(), equal to operator not equal to operator

Domain: Control Flow

Explanation:

The four searched names have in common, that they all have four letters.

Therefore len(d) == 4 fits the bill.

Evaluate the following Python arithmetic expression:

$(3 * (1 + 2) ** 2 - (2 ** 2) * 3)$

What is the result?

- a) 15
- b) 69
- c) 3
- d) 13

Answer: a) 15

Topics: addition operator subtraction operator, multiplication operator exponentiation operator, parentheses operator precedence

Domain: Fundamentals

Explanation:

The order of operator precedence here is:

Parentheses

Exponentiation operator

Multiplication operator

Subtraction operator

Try it yourself:

```
print((3 * (1 + 2) ** 2 - (2 ** 2) * 3)) # 15
print(3 * 3 ** 2 - (2 ** 2) * 3)      # 15
print(3 * 3 ** 2 - 4 * 3)              # 15
print(3 * (3 ** 2) - 4 * 3)           # 15
print((3 * 9) - 4 * 3)                 # 15
print(27 - 4 * 3)                      # 15
print(27 - (4 * 3))                    # 15
print(27 - 12)                         # 15
print(15)                              # 15
```

What is the expected output of the following code if the user enters 3 and 2?

```
x = int(input())
y = int(input())
x = x % y
x = x % y
y = y % x
print(y)
a) 2
b) 3
c) 1
d) 0
```

Answer: d) 0

Topics: input() int() modulus operator

Domain: Fundamentals

Explanation:

input() returns a string but the int() function casts them to an integer

Then there is a lot of the modulus operator but you just have to concentrate.

Try it yourself:

```
# x = int(input()) # Input: 3
# y = int(input()) # Input: 2
x, y = 3, 2        # Just for convenience
x = x % y
print(3 % 2) # 1
x = x % y
print(1 % 2) # 1
print(x)
print(y)
y = y % x
print(2 % 1) # 0
print(y)     # 0
```

You are writing a Python program that evaluates an arithmetic formula.
The formula is described as b equals a multiplied by negative one,
then raised to the second power,

where a is the value that will be input and b is the result.

```
a = eval(input('Enter a number for the equation: '))
```

Which of the following is a valid expression for the given requirement?

- a) `b = -(a) ** 2`
- b) `b = (a-) ** 2`
- c) `b = (-a) ** 2`
- d) `b = (a) ** -2`

Answer: c) `b = (-a) ** 2`

Topics: `eval()` `input()` exponentiation operator, operator precedence unary minus operator

Domain: Fundamentals

Explanation:

In the end this is mainly about operator precedence

The exponentiation operator has a higher precedence
than the unary minus operator

Try it yourself:

1. `# a = eval(input('Enter a number for the equation: '))`
2. `a = eval('7')` # Just for convenience
3. `print((-a) ** 2)` # 49
4. `print(-(a) ** 2)` # -49
5. `print(-a ** 2)` # -49
6. `print(-(a ** 2))` # -49

What is the output of the following snippet?

```
y = 2 + 3 * 5.
```

```
print(y)
```

- a) 17.0
- b) the snippet will cause an execution error
- c) 25.0
- d) 17

Answer: a) 17.0

Topics: arithmetic operators operator precedence float

Domain: Fundamentals

Explanation:

5. is the same as 5.0

and multiplication precedes addition.

What value will be assigned to the x variable?

```
z = 2
```

```
y = 1
```

```
x = y < z or z > y and y > z or z < y
```

- a) 1
- b) 0
- c) True
- d) False

Answer: c) True

Topics: operators relational operators operator precedence

Domain: Fundamentals

Explanation:

```
z = 7
y = 3
x = y < z or z > y and y > z or z < y
print(x)                # True
print(y < z or z > y and y > z or z < y) # True
print(3 < 7 or 7 > 3 and 3 > 7 or 7 < 3) # True
print(True or True and False or False) # True
print(True or (True and False) or False) # True
print(True or False or False)          # True
print((True or False) or False)        # True
print(True or False)                   # True
print(True)                            # True
```

The operators here are from three different groups.

"Comparisons, Identity, Membership operators", "Logical AND", "Logical OR".

The two comparison operators,

greater than operator and less than operator have the highest precedence.

Then the logical and operator has a higher precedence

than the logical or operators

What would you insert instead of ???

so that the program prints TRUE to the monitor?

```
w = 7
x = 3
y = 4
z = True
a = w + x * y
b = w + x / z

if ???:
    print('TRUE')
else:
    print('FALSE')
```

- a) $a < b$
- b) $a == b$
- c) $a > b$
- d) $a \leq b$

Answer: c) $a > b$

Topics: if else addition operator multiplication operator, division operator, operator precedence boolean

Domain: Control Flow

Explanation:

The operators here come from two different groups:

The group "Multiplication, Division, Floor division, Modulus"

has a higher precedence than the group

"Addition, Subtraction".

Meaning multiplication or division come before addition.

The other topic here is calculating with a boolean

If you calculate with a boolean then True will become 1 and False will become 0

The // operator:

- a) performs regular division
- b) Your answer is correct
- c) performs integer division
- d) does not exist

Answer: b) performs integer division

Topic: floor division operator

Domain: Fundamentals

Explanation:

Integer division is not fully correct.

Better is floor division operator:

If both operands are integers, the result is a floored integer.

But if at least one operand is a float, the result will be a floored float

Try it yourself:

1. `print(4 // 3)` # 1
2. `print(4.0 // 3)` # 1.0
3. `print(4 // 3.0)` # 1.0
4. `print(4.0 // 3.0)` # 1.0

How many stars will the following snippet print to the monitor?

`x = 16`

`while x > 0:`

`print('*')`

`x //= 2`

- a) three
- b) The code will enter an infinite loop.
- c) one
- d) five

Answer: d) five

Topics: while, greater than operator, print() with end parameter, floor-divide and assign operator

Domain: Control Flow

Explanation:

After each printing a floor division by 2 takes place.

And `1 // 2` is 0 and that's where the while loop ends.

What is the expected output of the following code?

`marks = [80, 70, 90, 90, 80, 100]`

`average = sum(marks) // len(marks)`

`grade = "`

`if 90 <= average <= 100:`

`grade = 'A'`

`elif 80 <= average < 90:`

`grade = 'B'`

`elif 70 <= average < 80:`

`grade = 'C'`

`elif 65 <= average < 70:`

`grade = 'D'`

`else:`

`grade = 'F'`

```
print(grade)
```

- a) D
- b) C
- c) A
- d) F
- e) B
- f) The code is erroneous.

Answer: e) B

Topics: if elif else list sum() len() chained comparison

Domain: Control Flow

Explanation:

Here you really have to do a little mental arithmetic.

$80 + 70 + 90 + 90 + 80 + 100 \rightarrow 510$

$510 // 6 \rightarrow 85$

85 is a the grade B

Analyze the following code fragments

that assign a boolean value to the variable even?

```
num = 42
```

```
# Code-1
```

```
if num % 2 == 0:
```

```
    even = True
```

```
else:
```

```
    even = False
```

```
# Code-2
```

```
even = True if num % 2 == 0 else False
```

```
# Code-3
```

```
even = num % 2 == 0
```

- a) All three are correct, but Code-3 is preferred.
- b) All three are correct, but Code-1 is preferred.
- c) Code-2 has a syntax error
because you cannot have True and False literals
in the conditional expression.
- d) Code-3 has a syntax error
because you attempt to assign a number to even
- e) All three are correct, but Code-2 is preferred.

Answer: a) All three are correct, but Code-3 is preferred.

Topics: if else modulus operator format()

Domain: Control Flow

Explanation:

They all work without a syntax error but Code-3 is preferred.

The condition $\text{num \% 2 == 0}$ evaluates to a boolean
and therefore that is all you need.

What is the expected output of the following code?

```
data = ['Peter', 'Paul', 'Mary']  
print(data[int(-1 / 2)])
```

- a) None of the above.
- b) Paul
- c) The code is erroneous.
- d) Peter
- e) Mary

Answer: d) Peter

Topics: list int() division operator

Domain: Data Collections

Explanation:

First the division takes place and -0.5 is the result.

The integer value of -0.5 is 0

And index 0 is 'Peter'

Try it yourself:

```
data = ['Peter', 'Paul', 'Mary']  
print(data[int(-1 / 2)]) # Peter
```

```
print(-1 / 2) # -0.5  
print(int(-0.5)) # 0
```

How many stars (*) will the following code output to the screen?

```
n = 1  
if n == 1:  
    print("*")  
if n == True:  
    print("***")  
if n == False:  
    print("****")  
a) two  
b) three  
c) six  
d) one
```

Answer: b) three

Topics: if equal to operator boolean implicit type casting

Domain: Control Flow

Explanation:

1 is equal to 1 and one star is printed.

If you compare an integer with a boolean,
the boolean True will be implicitly casted to the integer 1
and again 1 is equal to 1 and two more stars are printed.

How many stars will the following code print to the monitor?

```
x = 1  
while x < 10:  
    print('*')  
    x = x << 1  
a) four
```

- b) two
- c) eight
- d) one

Answer: a) four

Topics: while bit operator

Domain: Control Flow

Explanation:

Left shift by one, a classic way to double values.

Every value goes to the bit on its left and thereby doubles in value.

And 1, 2, 4, 8 are all smaller than 10 but not the 16

and therefore four stars will be printed.

Try it yourself:

```
x = 1
while x < 10:
    print('*')
    x = x << 1
    print('x:', x)      # 2 -> 4 -> 8 -> 16
    print('bin(x):', bin(x)) # 0b10->0b100->0b1000->0b10000
```

"""

1 -> 00001 << 1 -> 00010 (2)

2 -> 00010 << 1 -> 00100 (4)

4 -> 00100 << 1 -> 01000 (8)

8 -> 01000 << 1 -> 10000 (16)

"""

PCEP-30-02 1.5 – Perform Input/Output console operations

Topics

- the print() and input() functions
- the sep= and end= keyword parameters
- the int() and float() functions

Question and Answers

What will be the output of the following code when the user enters 2 and then 3?

```
x = input()
y = int(input())
print(x * y)
a) 6
b) 23
c) 222
d) 33
```

Answer: c) 222

Overall explanation

Note that we convert y to an integer, but we don't do the same with x. That means that x='2' (string) and y = 3. The multiplication of string '2' in Python by an integer 3 results in '222' (the string is repeated 3 times).

Section 2: Control Flow – Conditional Blocks and Loops (29%)

Objectives covered by the block (8 exam items)

PCEP-30-02 2.1 – Make decisions and branch the flow with the if instruction

Topics

- conditional statements: if, if-else, if-elif, if-elif-else
- multiple conditional statements
- nesting conditional statements

Question and Answers

Consider the following program to calculate a discount percentage:

```
day = input('Enter the day of the week:')
```

```
discount = 3
```

```
if day == 'Wednesday':
```

```
    discount += 5
```

```
elif day == 'Thursday':
```

```
    discount += 7
```

```
elif day == 'Saturday':
```

```
    discount += 10
```

```
elif day == 'Sunday':
```

```
    discount += 20
```

```
else:
```

```
    discount += 2
```

Which of the following inputs will get the user a discount of 5 %?

- a) Wednesday
- b) Saturday
- c) Sunday
- d) Friday
- e) Thursday

Answer: d) Friday

Topic: if, elif, else

Domain: Control Flow

Explanation: Friday is in none of the conditions and therefore the else clause gets executed.

And 3 plus 2 is 5

The ABC Video company needs a way to determine the cost that a customer will pay for renting a DVD.

The cost is dependent on the time of day the DVD is returned.

However, there are also special rates on Thursdays and Sundays.

The fee structure is shown in the following list:

The cost is \$1.59 per night.

If the DVD is returned after 8 PM, the customer will be charged an extra day.

If the video is rented on a Sunday,

the customer gets 30% off for as long as they keep the video.

If the video is rented on a Thursday,

the customer gets 50% off for as long as they keep the video.

You need to write code to meet the requirements.

```
# ABC Video, DVD Rental Calculator

ontime = input('Was the video returned before 8 pm? y or n').lower()
days_rented = int(input('How many days was the video rented?'))
day_rented = input('What day was the video rented?').capitalize()

cost_per_day = 1.59

if ontime XXX
    days_rented += 1
if day_rented YYY
    total = (days_rented * cost_per_day) * .7
elif day_rented ZZZ
    total = (days_rented * cost_per_day) * .5
else:
    total = (days_rented * cost_per_day)

print('Cost of the DVD rental is: $', total)
```

What should you insert instead of XXX, YYY and ZZZ?

- a) XXX -> == 'y':
YYY -> == 'Sunday':
ZZZ -> == 'Thursday':
- b) XXX -> == 'y':
YYY -> >= 'Sunday':
ZZZ -> >= 'Thursday':
- c) XXX -> == 'n':
YYY -> is 'Sunday':
ZZZ -> is 'Thursday':
- d) XXX -> == 'n':
YYY -> == 'Sunday':
ZZZ -> == 'Thursday':
- e) XXX -> != 'n':
YYY -> is 'Sunday':
ZZZ -> is 'Thursday':

Answer: d) XXX -> == 'n':
YYY -> == 'Sunday':
ZZZ -> == 'Thursday':

Topics: input() int() if elif else

Domain: Control Flow

Explanation:

At this company to return a video on time means to return it before 8 PM.

Otherwise one day is added to the amount of rented days.

If you return your video on a Sunday you get 30% off.

Meaning you only pay 70%.

And if you return your video on a Thursday, you pay only half (50%).

You are creating a Python script to evaluate input
and check for upper and lower case.

Code segment 1:

```
else:  
    print(name, 'is mixed case.')
```

Code segment 2:

```
else:  
    print(name, 'is lower case.')
```

Code segment 3:

```
name = input('Enter your name: ')
```

Code segment 4:

```
else:  
    print(name, 'is upper case.')
```

Code segment 5:

```
elif name.upper() == name:  
    print(name, 'is all upper case.')
```

Code segment 6:

```
if name.lower() == name:  
    print(name, 'is all lower case.')
```

Which four code segments should you use to develop the solution?

- a) Code segment 3
Code segment 6
Code segment 5
Code segment 1
- b) Code segment 3
Code segment 6
Code segment 5
Code segment 2
- c) Code segment 1
Code segment 3
Code segment 5
Code segment 6
- d) Code segment 3
Code segment 6
Code segment 5
Code segment 4

Answer: a) Code segment 3
Code segment 6
Code segment 5

Code segment 1

Topics: if elif else input() lower() upper()

Domain: Control Flow

Explanation:

First you need the input()

Then you need the if

Then you need the elif

There are three code segments with an else clause.

'lower' was in the if clause.

'upper' was in the elif clause.

That leaves 'mixed' for the else clause.

You are designing a decision structure

to convert a student's numeric grade to a letter grade.

The program must assign a letter grade as specified as followed:

90 through 100 -> A

80 through 89 -> B

70 through 79 -> C

65 through 69 -> D

0 through 64 -> F

For example, if the user enters a 90

the output should be Your letter grade is A

Likewise, if a user enters an 89

the output should be Your letter grade is B

```
# Letter Grade Converter
```

```
grade = int(input('Enter a numeric grade:'))
```

```
# Line-3
```

```
    letter_grade = 'A'
```

```
# Line-5
```

```
    letter_grade = 'B'
```

```
# Line-7
```

```
    letter_grade = 'C'
```

```
# Line-9
```

```
    letter_grade = 'D'
```

```
else:
```

```
    letter_grade = 'F'
```

```
print('Your letter grade is:', letter_grade)
```

Which of the following should you insert on

Line-3, Line-5, Line-7 and Line-9?

- a) if grade < 90: # Line-3
 elif grade < 80: # Line-5
 elif grade < 70: # Line-7
 elif grade < 65: # Line-9
- b) if grade > 90: # Line-3
 elif grade > 80: # Line-5
 elif grade > 70: # Line-7
 elif grade > 65: # Line-9
- c) if grade >= 90: # Line-3
 elif grade >= 80: # Line-5

```
elif grade >= 70: # Line-7
elif grade >= 65: # Line-9
d) if grade <= 90: # Line-3
    elif grade <= 80: # Line-5
    elif grade <= 70: # Line-7
    elif grade <= 65: # Line-9
```

Answer: c) if grade >= 90: # Line-3
elif grade >= 80: # Line-5
elif grade >= 70: # Line-7
elif grade >= 65: # Line-9

Topics: if elif else input() int()

Domain: Control Flow

Explanation:

It is 90 through 100 and therefore: grade >= 90

It is 80 through 89 and therefore: grade >= 80

It is 70 through 79 and therefore: grade >= 70

It is 65 through 69 and therefore: grade >= 65

And if the grade is 64 or below the else clause is executed.

What is the expected output of the following code?

```
data = [1, {}, (2,), (), {3}, [4, 5]]
points = 0
```

```
for i in range(len(data)):
    if type(data[i]) == list:
        points += 1
    elif type(data[i]) == tuple:
        points += 10
    elif type(data[i]) == set:
        points += 100
    elif type(data[i]) == dict:
        points += 1000
    else:
        points += 10000
```

```
print(points)
```

- a) 11121
- b) 10212
- c) 10221
- d) 11112
- e) 21102

Answer: a)

Topics: if, elif, else, type(), list, tuple, set, dict, equal to operator, add and assign operator

Domain: Control Flow

Explanation:

Every element of data gets tested in the for loop.

There is one integer, which executes the else clause.

There is one dictionary: {}

There is one set: {3}

There are two tuples: (2,) and ()

There is one list: [4, 5]

Consider the following code.

```
x = 42
y = 7
data = "I'm gonna make him an offer he can't refuse."
```

Which of the following expressions will evaluate to 19?

- a) 19 if None else x / y
- b) data.find('an') if data else None
- c) Your answer is incorrect
- d) 7 if len(data) > 19 else 6
- e) data.rfind('an') if data else None

Answer: b) data.find('an') if data else None

Topics: conditional expression (if else) find() rfind() len(), None division operator

Domain: Control Flow

Explanation:

The find() method is looking for the first occurrence of a string in a string and returns the index, where this string is starting.

In this case 'an' starts in data at index 19

rfind() looks from the right and will find the last occurrence.

In this case the 'an' in "can't" at index 32

None evaluates to False and therefore the else clause executes.

And yes, the string is longer than 19 but anyway 7 and 6 are both not 19

Try it yourself:

```
x = 42
y = 7
data = "I'm gonna make him an offer he can't refuse."
```

```
print(data.find('an') if data else None) # 19
print(19 if None else x / y)           # 6.0
print(data.rfind('an') if data else None) # 32
print(7 if len(data) > 19 else 6)      # 7
```

What will be the output when the user provides the number 0?

```
num = int(input('give a number: '))
if num > 0:
    if num < 5:
        print('1')
    else:
        print('2')
else:
    if num < -1:
        print('3')
    else:
        print('4')
```

- a) 3
- b) 2
- c) 1
- d) 4

Answer: 4

Overall explanation

The first condition (num > 0) is not satisfied for input 0, so we go to the else part. Inside, the inner condition (num < - 1) isn't satisfied either, so we end up in the nested else part, which prints 4.

What will be the output when the user provides the number 5?

```
num = int(input('give a number: '))
if num > 3:
    if num < 6:
        print('A')
    else:
        print('B')
else:
    print('C')
a) C
b) B
c) Python will show an error
d) A
```

Answer: d) A

Overall explanation

The first condition (num > 3) is satisfied for input 5. Then, the nested condition (num < 6) is also satisfied for input 5. That's why A is printed to the output.

PCEP-30-02 2.2 – Perform different types of iterations

Topics

- the pass instruction
- building loops with while, for, range(), and in
- iterating through sequences
- expanding loops with while-else and for-else
- nesting loops and conditional statements
- controlling loop execution with break and continue

Question and Answers

What does the following code do?

```
def a(b, c, d):
    pass
a) Defines a function, which passes its parameters through.
b) Defines a function, which does nothing.
c) Defines an empty class.
d) Defines a list and initializes it.
e) None of the above.
```

Answer: b) Defines a function, which does nothing

Topics: def pass

Domain: Functions and Exceptions

Explanation:

def a(b, c, d): defines the function
and pass just does nothing.

You can use that, when you know, that you will need a function,
but want to write the function body later on.

What will be the output of the following code?

```
for i in range(5):  
    print(i, end='-')
```

- a) 0-1-2-3-4-
- b) Python will show an error
- c) 0-1-2-3-4-5-
- d) 1-2-3-4-5-

Answer: a) 0-1-2-3-4

Overall explanation

range(5) will generate values from 0 to 4 inclusive. In the print call, we set end to '-', which means that we will finish each print statement with a dash. This gives 0-1-2-3-4-.

What is the expected output of the following snippet?

```
s = 'python'  
for i in range(len(s)):  
    i = s[i].upper()  
print(s, end="")
```

- a) P y t h o n
- b) Python
- c) PYTHON
- d) The code is erroneous.
- e) P Y T H O N
- f) python

Answer: f) python

Topics: for range() len() upper() string indexing

Domain: Control Flow

Explanation:

A string is immutable.

You can not change it, even if you tried.

Here it is not even tried

otherwise the code would cause a TypeError

How many stars will the following code print to the monitor?

```
i = 0  
while i < i + 2:  
    i += 1  
    print('*')  
else:  
    print('*')
```

- a) The snippet will enter an infinite loop.
- b) zero
- c) one
- d) two

Answer: a) The snippet will enter an infinite loop

Topics: while else (nobreak)

Domain: Control Flow

Explanation:

i gets incremented inside the while loop,

BUT i will always be smaller than i + 2

Therefore the while condition will always be True

and we build ourselves a nice infinite loop.

How many stars will the following snippet print to the monitor?

```
for i in range(1):
    print('*')
else:
    print('*')
a) three
b) zero
c) two
d) one
```

Answer: c) two

Topics: for else (nobreak)

Domain: Control Flow

Explanation:

There will be just one iteration.

But because there is no break inside the for loop,

the else clause will execute and the second star will be printed.

Try it yourself:

```
for i in range(1):
    print('i:', 1) # 1
    print('*')    # *
else:
    print('*')    # *
```

How many stars will the following snippet send to the console?

```
for i in range(-1, 1):
    print("*")
a) three
b) one
c) two
```

Answer: c) two

Topics: for range()

Domain: Control Flow

Explanation:

In the first iteration i will be -1

and in the second iteration i will be 0

Therefore two stars will be printed.

How many times will the x sign be printed?

```
for a in range(1,3):
    for b in range(1, 8):
        if a == b:
            print('x')
a) 8
```

- b) 3
- c) 5
- d) 2

Answer: d) 2

Overall explanation

In the outer loop, range(1, 3) generates two values: 1 and 2. In the nested loop, range(1, 8) generates 7 values: from 1 to 7. The only two times when value a from the outer loop and value b from the inner loop are equal are (a=1, b=1) and (a=2, b=2). That's why you will see x printed twice.

What will be the output of the following code?

```
for a in range(1, 3):
    for b in range(1, 3):
        if a > b:
            continue
        else:
            print(a * b)
```

- a) 1
2
4
9
- b) 1
2
2
6
9
- c) 1
2
4
- d) 1
2
2
4

Answer: c)

1
2
4

Overall explanation

range(1, 3) generates two values: 1 and 2. You can see a nested loop here which also uses range(1, 3). This gives us four combinations of outer and inner loop:

a = 1, b = 1 => a * b printed (1 * 1 = 1)

a = 1, b = 2 => a * b printed (1 * 2 = 2)

a = 2, b = 1 => a > b so nothing is printed

a = 2, b = 2 => a * b printed (2 * 2 = 4)

How many stars will the following snippet print to the monitor?

```
i = 0
while i <= 5:
    i += 1
    if i % 2 == 0:
        break
```

```
print('*')
```

- a) two
- b) three
- c) one
- d) zero

Answer: c) one

Topics: while break

Domain: Control Flow

Explanation:

$i \% 2 == 0$ is known as test for even numbers.

In the first iteration i is an odd 1

and therefore the break does not get triggered
and a star gets printed.

In the second iteration i is an even 2

the break gets triggered and there is no more stars.

How many stars will the following code send to the monitor?

```
x = 0
while x < 6:
    x += 1
    if x % 2 == 0:
        continue
    print('*')
```

- a) two
- b) three
- c) zero
- d) one

Answer: b) three

Topics: while if continue

Domain: Control Flow

Explanation:

When x is 2, 4 and 6 the if condition is True
and the continue gets triggered.

The iteration ends directly and the print() function doesn't get executed.

When x is 1, 3 and 5 the if condition is False
the continue does not get triggered

and those three times the print() function gets executed.

Therefore there will be three stars printed.

Try it yourself:

```
x = 0
while x < 6:
    print('1. x:', x)    # 0 -> 1 -> 2 -> 3 -> 4 -> 5
    x += 1
    print('2. x:', x)    # 1 -> 2 -> 3 -> 4 -> 5 -> 6
    if x % 2 == 0:
        print('x in if:', x) # 2 -> 4 -> 6
        continue
    print('x behind if:', x) # 1 -> 3 -> 5
    print('*')
```

```
""""
*
*
*
""""
```

Which one of the lines should you put in the snippet below to match the expected output?

Expected output:

adam_smit

Code:

```
for ch in "adam_smit@openedg.org":
```

```
    if ch == "@":
```

```
        # insert code here
```

```
    print(ch, end="")
```

- a) continue
- b) exit
- c) break
- d) print()

Answer: c) break

Topics: break for with strings if equal to operator

Domain: Control Flow

Explanation:

The for loop with a string will assign one character after the other to ch

When the @-character is assigned the loop needs to

stop in order to end the printing of the characters.

That is what the break keyword is for.

You are developing a Python application

for an online product distribution company.

You need the program to iterate through a list of products

and escape when a target product ID is found.

```
productIdList = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
index = 0
```

```
??? index < 10:
```

```
    print(productIdList[index])
```

```
    if productIdList[index] == 6:
```

```
        ???
```

```
    else:
```

```
        index += 1
```

What would you insert instead of ??? and ???

- a) break
- while
- b) while
- break
- c) while

- for
- d) for
- while
- e) for
- break
- f) if
- break

Answer: b) while
 break

Topics: list while for break if else, equal to operator

Domain: Control Flow

Explanation:

The while loop is the loop, that works with a condition.

The break keyword ends a loop.

Which of the following code snippets will print all prime numbers between 2 and 100 to the monitor?

- a) num = 2
is_prime = True
while num <= 100:
 for i in range(2, num):
 if num % i == 0:
 is_prime = False
 break
 if is_prime == True:
 print(num)
 num += 1
- b) num = 2
while num <= 100:
 is_prime = True
 for i in range(2, num):
 if num % i == 0:
 is_prime = False
 break
 if is_prime == True:
 print(num)
- c) num = 2
while num <= 100:
 is_prime = True
 for i in range(2, num):
 if num % i == 0:
 is_prime = False
 break
 if is_prime == False:
 print(num)
 num += 1
- d) num = 2
while num <= 100:
 is_prime = True
 for i in range(2, num):
 if num % i == 0:

```
        is_prime = False
        break
    if is_prime == True:
        print(num)
    num += 1
```

Answer: d) num = 2

```
while num <= 100:
    is_prime = True
    for i in range(2, num):
        if num % i == 0:
            is_prime = False
            break
    if is_prime == True:
        print(num)
    num += 1
```

Topics: while break boolean for range(), modulus operator equal to operator if

Domain: Control Flow

Explanation:

One answer does not increment the number.

One answer has the is_prime flag outside of the outer loop.

One answer checks, if it is False the the number is a prime number.

And one answer does it all the right way.

Consider the following code.

```
nums = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
x = 0
while x < 10:    # Line 3
    print(nums(x)) # Line 4
    if nums[x] = 7: # Line 5
        break    # Line 6
    else:        # Line 7
        x += 1    # Line 8
```

You want to print the numbers 1 to 7 to the monitor.

But the code does not work.

What do you have to change?

Choose two.

- a) print(nums[x]) # Line 4
- b) if nums[x] == 7: # Line 5
- c) while (x < 10): # Line 3
- d) x = x + 1 # Line 8

Answer:

- a) print(nums[x]) # Line 4
- b) if nums[x] == 7: # Line 5

Topics: while break if else list equal to operator

Domain: Control Flow

Explanation:

Line 3 and Line 8 work just fine.

Line 4 needs brackets for the list indexing.

Line 5 needs the equal to operator

Try it yourself:

```
nums = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
x = 0
while x < 10:    # Line 3
    print(nums[x]) # Line 4
    if nums[x] == 7: # Line 5
        break      # Line 6
    else:          # Line 7
        x += 1     # Line 8
```

Which one of the lines should you put in the snippet below to match the expected output?

Expected output:

1245

Code:

```
c = 0
while c < 5:
    c = c + 1
    if c == 3:
        # enter code here
    print(c, end="")
```

- a) exit
- b) continue
- c) print()
- d) break

Answer: b) continue

Topics: continue while if comparison operators

Domain: Control Flow

Explanation:

You want to print every number except the number 3

Therefore when c is equal to 3 you want to end

that iteration of the loop directly before the number is printed.

That is what the continue keyword is for.

Consider the following code.

```
for i in range(5, 0, ???):
    print(i, i, i, i, i)
```

What would you insert instead of ??? so that the program prints the following pattern to the monitor?

5 5 5 5 5

4 4 4 4 4

3 3 3 3 3

2 2 2 2 2

1 1 1 1 1

a) 0

b) -1

c) None

d) 1

Answer: b) -1

Topics: range() for

Domain: Control Flow

Explanation:

By inserting -1 in place of ???,

the range() function will iterate from 5 down to 1 in steps of -1, resulting in the desired pattern being printed to the monitor.

Inserting 0 in place of ??? would cause a ValueError

Inserting None in place of ??? would cause a TypeError

Inserting 1 in place of ??? would cause the range() function to return an empty range object and nothing would get printed, because if the third argument (the step) is a positive number, the first argument (the start) needs to be lesser than the second argument (the end).

How many @ signs (@) will be printed after the following code snippet?

```
a = 0
while a <= 5:
    a += 2
    print('@')
a) 2
b) 3
c) 1
d) 4
```

Answer: b) 3

Overall explanation

When we 1st enter the loop, a = 0. We add 2 (a = 2) and print @.

When we enter the loop for the 2nd time: a = 2. We add 2 (a = 4) and print @.

When we enter the loop for the 3rd time: a = 4. We add 2 (a = 6) and print @.

At this point, a = 6, which does not satisfy the condition of the loop (a <= 5), so we terminate the loop.

How many percentage (%) signs will be printed after the following code snippet?

```
for i in range(2):
    print('%')
else:
    print('%')
a) 1
b) 2
c) 3
d) 0
```

Answer: c) 3

Overall explanation

range(2) generates two values: 0 and 1. The values themselves aren't used in any way in the loop, but instead we print @ for each of them. This gives us two @ signs. The third @ sign comes from the else part. Remember that else statements work differently with loops than with if statements: they are always executed after the loop successfully finishes its execution.

Section 3: Data Collections – Tuples, Dictionaries, Lists, and Strings (25%)

Objectives covered by the block (7 exam items)

PCEP-30-02 3.1 – Collect and process data using lists

Topics

- constructing vectors
- indexing and slicing
- the len() function
- list methods: append(), insert(), index(), etc.
- functions: len(), sorted()
- the del instruction
- iterating through lists with the for loop
- initializing loops
- the in and not in operators
- list comprehensions
- copying and cloning
- lists in lists: matrices and cubes

Question and Answers

What is the expected output of the following code?

```
data = [[42, 17, 23, 13], [11, 9, 3, 7]]
res = data[0][0]
for da in data:
    for d in da:
        if res > d:
            res = d
print(res)
```

- a) 3
- b) 13
- c) The code is erroneous.
- d) 42

Answer: a) 3

Topics: list for if

Domain: Control Flow

Explanation:

This code snippet looks for the lowest element in the two dimensional list data

In the beginning the first number data[0][0] gets taken as possible result.

In the inner for loop every number is compared to the possible result.

If one number is lower it becomes the new possible result.

And in the end the result is the lowest number.

Try it yourself:

```
data = [[42, 17, 23, 13], [11, 9, 3, 7]]
res = data[0][0]
print(res) # 42
```

```
for da in data:
    for d in da:
        if res > d:
            print('d: ', d) # 17 -> 13 -> 11 -> 9 -> 3
            res = d
print(res) # 3
```

Which of the following sentences correctly describes the output of the below Python code?

```
data = [4, 2, 3, 2, 1]
res = data[0]
```

```
for d in data:
    if d < res:
        res = d
```

```
print(res)
```

- a) res is the average of all the number in the list.
- b) res is the smallest number in the list.
- c) None of the above.
- d) res is the largest number in the list.
- e) res is the sum of all the number in the list.

Answer: b) res is the smallest number in the list

Topics: for if list

Domain: Control Flow

Explanation:

Classic way to find the smallest number.

Take the first element as possible result.

Compare the next number with the possible result.

If the next number is smaller,

it becomes the new possible result and so forth.

In the end the result is the smallest number.

Consider the following code.

```
data = ['Peter', 'Paul', 'Mary', 'Jane']
res = 0
```

Which of the following code snippets will expand the code, so that 100 will be printed to the monitor?

Choose two.

- a)

```
for i in ('Peter', 'Steve', 'Jane'):
    if i not in data:
        res += 100

print(res)
```
- b)

```
for i in ('Peter', 'Steve', 'Jane'):
    if i not in data:
        res += 50

print(res)
```
- c)

```
for i in ('Peter', 'Steve', 'Jane'):
    if i in data:
        res += 50

print(res)
```

```
d) for i in ('Peter', 'Steve', 'Jane'):
    if i in data:
        res += 100
    print(res)
```

Answer: a) and c)

```
for i in ('Peter', 'Steve', 'Jane'):
    if i not in data:
        res += 100
    print(res)
for i in ('Peter', 'Steve', 'Jane'):
    if i in data:
        res += 50
    print(res)
```

Topics: list for if membership operator not

Domain: Control Flow

Explanation:

Only Steve is not in data

Therefore we need the += 100 there.

Peter and Jane are both in data

Therefore we need the += 50 there.

How many stars will the following snippet print to the monitor?

```
data = [[x for x in range(y)] for y in range(3)]
for d in data:
    if len(d) < 2:
        print('*')
```

- a) zero
- b) two
- c) three
- d) one

Answer: b) two

Topics: list, comprehension, if, len()

Domain: Control Flow

Explanation:

This does the same as the list comprehension:

1. data = []
2. for y in range(3):
3. temp = []
4. for x in range(y):
5. temp.append(x)
6. data.append(temp)
7. print(data) # [], [0], [0, 1]

The list comprehension produces a two-dimensional list.

The outer list has three elements.

Each of which is as long as its index: [], [0] and [0, 1]

The if in the for loop checks that length.

Two of the three are smaller than 2 and therefore two stars are printed.

How many stars will the following code print to the monitor?

```
data = [[x for x in range(3)] for y in range(3)]
```

```
for i in range(3):
    for j in range(3):
        if data[i][j] % 2 != 0:
            print('*')
```

- a) three
- b) nine
- c) six
- d) zero

Answer: a) three

Topics: list comprehension for if list indexing, modulus operator not equal to operator

Domain: Control Flow

Explanation:

The list comprehension produces a two-dimensional list

The three elements of the outer list also each have three elements:

the numbers 0, 1, 2

$x \% 2 \neq 0$ tests, if x is odd.

The two for loops make it happen, that every of the nine elements is tested.

There is three times the number 1

Those are the odd ones.

Therefore three stars get printed.

Try it yourself:

```
data = [[x for x in range(3)] for y in range(3)]
print(data) # [[0, 1, 2], [0, 1, 2], [0, 1, 2]]
for i in range(3):
    for j in range(3):
        if data[i][j] % 2 != 0:
            print('*') # * * *
```

What is the expected output of the following code?

```
my_list = [[3-i for i in range(3)] for j in range(3)]
```

```
result = 0
```

```
for i in range(3):
    result += my_list[i][i]
```

```
print(result)
```

- a) 2
- b) 4
- c) 7
- d) 6

Answer: d) 6

Topics: list, comprehension, list indexing, for

Domain: Control Flow

Explanation:

The list comprehension produces a two-dimensional list.

The three elements of the outer list also each have three elements:

the numbers 3, 2, 1

The for loop will add together

the first element of the first list 3
the second element of the second list 2
and the third element of the third list 1
 $3 + 2 + 1 \rightarrow 6$

What is the expected output of the following code?

```
print(len([i for i in range(0, -2)]))
```

- a) 1
- b) 3
- c) 0
- d) 2

Answer: c) 0

Topics: list comprehension, range(), len()

Domain: Control Flow

Explanation:

The "problem" here is the range(0, -2)

The start value 0 has to be lower than the end value -2

Otherwise range() does not return an element.

How many elements will the following list contain?

```
data = [i for i in range(-1, 2)]
```

- a) three
- b) one
- c) two
- d) four
- e) zero

Answer: a) three.

Topics: list comprehension, for

Domain: Control Flow

Explanation:

In data is going to be a list with the element -1, 0 and 1

The list comprehension is not necessary if I does not get changed.

The start of range() is inclusive -1

and the end is exclusive, meaning 2 is excluded.

How many elements does the L list contain?

```
L = [i for i in range(-1, -2)]
```

- a) two
- b) zero
- c) three
- d) one

Answer: b) zero

Topic: list comprehension

Domain: Data Collections

Explanation:

range(-1, -2) has no element

and therefore the list L will be empty.

How many elements does the my_list list contain?

```
my_list = [0 for i in range(1, 3)]
```

- a) three
- b) one
- c) two

Answer: c) two

Topic: list comprehension

Domain: Data Collections

Explanation:

range(1, 3) will be 1 & 2 and therefore

the for loop will iterate twice.

Both times 0 will be appended to the list.

What will be the output after the following code snippet?

```
temp = [[3 - i for i in range(2)] for j in range(3)]
print(temp)
```

- a) [3, 2, 3, 2, 3, 2]
- b) Python will show an error
- c) [[3, 2], [3, 2], [3, 2]]
- d) [[[3], [2]], [[3], [2]], [[3], [2]]]

Answer: c) [[3, 2], [3, 2], [3, 2]]

Overall explanation

[3 - i for i in range(2)] translates into [3, 2] because range(2) generates value 0 and 1. This gives the first element of 3 - 0 = 3, and the second element of 3 - 1 = 2.

The outer list comprehension generates three such lists [3, 2] using range(3) (which generates numbers 0, 1, 2).

What is the expected output of the following code?

```
data1 = (1, 2)
data2 = (3, 4)
[print(sum(x)) for x in [data1 + data2]]
```

- a) 4
- b) Nothing gets printed.
- c) The code is erroneous.
- d) 10

Answer: d) 10

Topics: tuple sum() list comprehension, plus operator tuple concatenation

Domain: Data Collections

Explanation:

data1 and data2 will get concatenated to one tuple

That leaves a list comprehension with a list of just one element,

the tuple (1, 2, 3, 4)

The sum() function adds the elements of a tuple and returns the sum.

sum((1, 2, 3, 4)) will be 10 which will be printed.

What is the expected output of the following code?

```
data = [
    [1, 2, 3, 4],
    [5, 6, 7, 8],
    [9, 10, 11, 12],
    [13, 14, 15, 16]
]
for i in range(0, 4):
    print(data[i].pop(), end=' ')
```

- a) 1 2 3 4
- b) 1 5 9 13
- c) 4 8 12 16
- d) 13 14 15 16

Answer: c) 4 8 12 16

Topics: list for range() pop() print() with end parameter

Domain: Data Collections

Explanation:

data is a two dimensional list.

In every iteration of the for loop,

pop() removes the last element of the corresponding inner list.

That last element gets printed.

The default value of the print() functions end parameter is the line feed.

That gets overwritten here by a single space character.

What is the expected output of the following code?

- ```
1. def func(item):
2. item += [1]
3.
4. data = [1, 2, 3, 4]
5. func(data)
6. print(len(data))
```

- a) 4
- b) 5
- c) 2
- d) The code is erroneous.

Answer: b) 5

Topic: #list #plus operator list concatenation #mutable argument

Domain: Functions and Exceptions

Explanation: # [1, 2, 3, 4] + [1] -> [1, 2, 3, 4, 1]

Inside the function a list concatenation by plus operator takes place.

The plus operator merges the elements of the second list into the first list.

append() would insert the list itself into the other list.

---

What is the expected output of the following code?

```
fruits1 = ['Apple', 'Pear', 'Banana']
fruits2 = fruits1
fruits3 = fruits1[:]
```

```
fruits2[0] = 'Cherry'
fruits3[1] = 'Orange'
```

```
res = 0
```

```
for i in (fruits1, fruits2, fruits3):
 if i[0] == 'Cherry':
 res += 1
 if i[1] == 'Orange':
 res += 10
```

```
print(res)
```

- a) 11
- b) 22
- c) 12
- d) 0

Answer: C) 12

Topics: for, copy vs. reference, list slicing, list indexing

Domain: Data Collections

Explanation:

A list is a mutable data type.

Assigning it will create a reference.

Slicing it from start to end [:] will create a copy.

Therefore Cherry goes in fruits2 and thereby also in fruits1

Orange only goes in fruits3

After updates, the lists will become:

List1: ['Cherry', 'Pear', 'Banana']

List2: ['Cherry', 'Pear', 'Banana']

List3: ['Apple', 'Orange', 'Banana']

---

Take a look at the snippet

and choose one of the following statements which is true:

```
nums = []
vals = nums
vals.append(1)
```

- a) vals is longer than nums
- b) nums is longer than vals
- c) nums and vals are of the same length

Answer: c) nums and vals are of the same length.

Topics: list referencing list.append()

Domain: Data Collections

Explanation:

A list is a mutable data type.

Assigning a mutable data type creates a reference to the same object.

vals and nums will point to the same object in the memory

and when you change one you automatically change the other, too.

---

What is the expected output of the following code?

```
nums = [1, 2, 3]
data = ('Peter',) * (len(nums) - nums[::-1][0])
print(data)
```

- a) ()
- b) The code is erroneous.
- c) PeterPeter
- d) ('Peter', 'Peter')
- e) ('Peter')

Answer: 2) ()

Topics: len(), list, tuple, list slicing, list indexing, multiply operator tuple concatenation

Domain: Data Collections



Explanation:

The length of nums is 3

[::-1] reverses the list and then 3 is the first index.

3 minus 3 makes 0

The trailing comma marks the tuple with one element.

Otherwise it would be a string with additional parentheses.

Any tuple multiplied by 0 becomes an empty tuple.

---

What is the expected output of the following code?

```
data = [1, 2, 3, 4, 5, 6]
```

```
for i in range(1, 6):
 data[i - 1] = data[i]
```

```
for i in range(0, 6):
 print(data[i], end=' ')
```

- a) 1 2 3 4 5 6
- b) 2 3 4 5 6 6
- c) 1 1 2 3 4 5
- d) 2 3 4 5 6 1

Answer: b) 2 3 4 5 6 6

Topics: for range() print() with end parameter list indexing

Domain: Data Collections

Explanation:

In the first for loop, the five last values of data  
get written to the first five indexes of data

The value of the last index stays the same.

range(1, 6) will produce the numbers 1, 2, 3, 4, 5

And because the first of these number is 1 we need the data[i - 1] to start at index 0

The old value at index 1 is 2

That gets written at index 0

And so forth ...

Try it yourself:

```
data = [1, 2, 3, 4, 5, 6]
```

```
for i in range(1, 6): # 1 -> 2 -> 3 -> 4 -> 5
 data[i - 1] = data[i]
```

```
print(data) # [2, 3, 4, 5, 6, 6]
```

```
This is just for output:
```

```
for i in range(0, 6):
 print(data[i], end=' ') # 2 3 4 5 6 6
```

---

What is the expected output of the following code?

```
numbers = [1, 2, 3, 4, 5]
```

```
nums = numbers[2:]
```

```
print(nums)
```

- a) [3, 4, 5]
- b) [2]

- c) The program will cause an error
- d) [2, 3, 4, 5]

Answer: a) [3, 4, 5]

Topic: list slicing

Domain: Data Collections

Explanation:

[start:stop]

The stop is not given which means the last element is included.

The start is index 2 (inclusive) and therefore the result is [3, 4, 5]

---

Take a look at the snippet and choose the true statement:

```
nums = [1, 2, 3]
vals = nums
del vals[:]
```

- a) nums is longer than vals
- b) the snippet will cause a runtime error
- c) nums and vals have the same length
- d) vals is longer than nums

Answer: nums and vals have the same length

Topics: list reference del list indexing

Domain: Data Collections

Explanation:

A list is a mutable data type.

Assigning a mutable data type creates a reference to the same object.

vals and nums will point to the same object in the memory

and when you change one you automatically change the other, too.

Try it yourself:

1. nums = [1, 2, 3]
2. vals = nums
3. del vals[:]
4. print(nums) # []
5. print(vals) # []

---

Consider the following list.

data = [1, 5, 10, 19, 55, 30, 55, 99]

Which of the code snippets below would produce a new list like the following?

[1, 5, 10, 99]

- a) data.pop(5)  
data.remove(19)  
data.remove(55)  
data.remove(55)
- b) None of the above
- c) data.pop(1)  
data.pop(3)  
data.pop(4)  
data.pop(6)
- d) data.pop(5)  
data.pop(19)  
data.pop(55)

```
e) data.remove(5)
 data.remove(19)
 data.remove(55)
```

Answer: a) data.pop(5)  
data.remove(19)  
data.remove(55)  
data.remove(55)

Topics: list pop() remove()

Domain: Data Collections

Explanation:

list.pop([i])

The index is optional.

If the index is given pop()

removes and returns the element at the given index.

The default index is -1

Meaning that the last index is removed and returned.

data.pop(5) removes the value 30

The remove() method removes the first occurrence  
of the element with the specified value.

data.remove(19)

data.remove(55)

data.remove(55)

is exactly what is needed.

---

What is the expected output of the following code?

```
data = [1, 2, [3, 4], [5, 6], 7, [8, 9]]
count = 0
```

```
for i in range(len(data)):
 if type(data[i]) == list:
 count += 1
```

```
print(count)
```

- a) The code is erroneous.
- b) 9
- c) 6
- d) 3

Answer: d) 3

Topics: list for range() len() if type() equal to operator

Domain: Control Flow

Explanation:

The snippet checks every element of the list, if it itself is a list  
and counts the occurrences.

There are three lists inside of the list data: [3, 4], [5, 6] and [8, 9]

---

What is the output of the following snippet?

```
my_list = [1, 2]
```

```
for v in range(2):
 my_list.insert(-1, my_list[v])
```

```
print(my_list)
a) [1, 1, 2, 2]
b) [1, 2, 2, 2]
c) [1, 2, 1, 2]
d) [1, 1, 1, 2]
```

Answer: d) [1, 1, 1, 2]

Topics: list for range() list.insert()

Domain: Data Collections

Explanation:

The index -1 represents the last element

and therefore the new elements will be inserted before the last element.

In the first iteration the element with the index 0 will be inserted.

The value of that index is 1 and therefore the list will be: [1, 1, 2]

In the second iteration the element with the index 1 will be inserted.

The value of that index is NOW also 1 and therefore the list will be:

[1, 1, 1, 2]

Try it yourself:

```
my_list = [1, 2]
```

```
for v in range(2):
 my_list.insert(-1, my_list[v])
```

```
print(my_list) # [1, 1, 1, 2]
```

# The same without for loop:

```
my_list_2 = [1, 2]
my_list_2.insert(-1, my_list_2[0])
print(my_list_2) # [1, 1, 2]
my_list_2.insert(-1, my_list_2[1])
print(my_list_2) # [1, 1, 1, 2]
```

---

The second assignment:

```
vals = [0, 1, 2]
vals[0], vals[1] = vals[1], vals[2]
```

- a) extends the list
- b) doesn't change the list's length
- c) shortens the list

Answer: b) doesn't change the list's length

Topics: list indexing multiple assignment

Domain: Data Collections

Explanation:

The value of index 0 will be overridden with the value of index 1

and the value of index 1 will be overridden with the value of index 2

Try it yourself:

```
vals = [0, 1, 2]
vals[0], vals[1] = vals[1], vals[2]
print(len(vals)) # 3
print(vals) # [1, 2, 2]
```

---

What is the expected output of the following code?

```
data = ['abc', 'def', 'abcde', 'efg']
print(max(data))
```

a) efg  
b) abcde  
c) abc  
d) The code is erroneous.  
e) def  
f) None of the above.

Answer: a) efg

Topics: list max()

Domain: Data Collections

Explanation:

Python's built-in function max() is given a list with strings.

max() will look for the string, which first character has the highest Unicode character number, which in this case is efg

because e has the highest Unicode character number.

Try it yourself:

```
data = ['abc', 'def', 'abcde', 'efg']
print(max(data)) # efg

print(ord('a')) # 97
print(ord('d')) # 100
print(ord('e')) # 101
```

---

What will happen with the list after the following operation?

```
nums = [1, 2, 3]
nums[0], nums[2] = nums[2], nums[0]
```

a) The list will not change at all  
b) The list will be longer  
c) The list will be effectively reversed  
d) The list will be shorter

Answer: c) The list will be effectively reversed

Overall explanation

The expression on line two swaps the values of the first and the third element, so we'll get [3, 2, 1]. This is the original list reversed.

---

What will be the output of the following code?

```
def check(x):
 if (x[0] == x[-1]):
 return True
 else:
 return False

print(check([10, 20, 30, 40, 10]))
print(check([40, 20, 30, 40, 10]))
```

a) True  
False

- b) False  
False
- c) True  
True
- d) False  
True

Answer: a) True

False

Overall explanation

The function checks whether the first (x[0]) and last (x[-1]) elements are equal. It's True for the first list and False for the second list.

---

Which of the following statement(s) is/are true?

```
list1 = [1, 2, 3]
```

```
list2 = list1[-3:-1]
```

- a) list1 and list2 have the same length
- b) list2 has fewer elements than list1
- c) list2 is longer than list1
- d) list1 and list2 are two different lists

Answer: b) and d)

b) list2 has fewer elements than list1

d) list1 and list2 are two different lists

Overall explanation

list1 = [1, 2, 3]. Now, on the second line, the expression list1[-3:-1] means: start with the third element from the back (inclusive), which is 1. and take all elements until the last element (**exclusive!**). This gives us [1, 2]. In the end, list1 = [1, 2, 3] and list2 = [1, 2].

---

What is the expected output of the following code?

```
data = {'Peter': 30, 'Paul': 31}
```

```
print(list(data.keys()))
```

- a) ['Peter': 30, 'Paul': 31]
- b) ('Peter', 'Paul')
- c) ['Peter', 'Paul']
- d) ('Peter': 30, 'Paul': 31)

Answer: c) ['Peter', 'Paul']

Topics: dictionary list() keys()

Domain: Data Collections

Explanation:

The key() method returns the keys() in a dict\_keys object.

list() turns that into a list

---

What is the expected output of the following code?

```
def get_names():
```

```
 names = ['Peter', 'Paul', 'Mary', 'Jane', 'Steve']
```

```
 return names[2:]
```

```
def update_names(names):
```

```
 res = []
```

```
 for name in names:
```

```
 res.append(name[:3].upper())
 return res
```

```
print(update_names(get_names()))
```

- a) ['MA', 'JA', 'ST']
- b) ['JAN', 'STE']
- c) ['JA', 'ST']
- d) ['MAR', 'JAN', 'STE']

Answer: d) ['MAR', 'JAN', 'STE']

Topics: def list slicing for append() upper()

Domain: Functions and Exceptions

Explanation:

The slicing in get\_names() is a list slicing: names[2:]

It has a start of 2 and no end.

That means, the slicing goes from the second index (inclusive) to the end.

That will be ['Mary', 'Jane', 'Steve']

That list will be passed to the update\_names() function.

The other slicing inside the for loop is a string slicing,

that will slice every element of the list

name[:3] will slice from the start to the index 3 (exclusive).

Meaning it takes the first three letters.

The upper() method will then convert those letters to uppercase letters.

---

## PCEP-30-02 3.2 – Collect and process data using tuples

### Topics

- tuples: indexing, slicing, building, immutability
- tuples vs. lists: similarities and differences
- lists inside tuples and tuples inside lists

### Question and Answers

What is the expected output of the following code?

```
x = (1, 4, 7, 9, 10, 11)
y = {2: 'A', 4: 'B', 6: 'C', 8: 'D', 10: 'E', 12: 'F'}
res = 1
for z in x:
 if z in y:
 res += z
print(res)
```

- a) 14
- b) 15
- c) 22
- d) 6

Answer: b) 15

Topics: tuple dictionary for if

Domain: Control Flow

Explanation:

The for loop iterates through the tuple x

The if statement checks, whether one of the elements of the tuple also is in index of the dictionary y

This is the case for 4 and for 10

Therefore 4 and 10 are added to the res variable, which is already 1 in the beginning:

$1 + 4 + 10 \rightarrow 15$

Try it yourself:

```
x = (1, 4, 7, 9, 10, 11)
y = {2: 'A', 4: 'B', 6: 'C', 8: 'D', 10: 'E', 12: 'F'}
res = 1
for z in x:
 print('z in for:', z)
 # z in for: 1
 # z in for: 4
 # z in for: 7
 # z in for: 9
 # z in for: 10
 # z in for: 11
 if z in y:
 print('z in if:', z)
 # z in if: 4
 # z in if: 10
 res += z
print(res) # 15
```

---

What is the expected output of the following code?

```
x = (1, 4, 7, 9, 10, 11)
y = {2: 'A', 4: 'B', 6: 'C', 8: 'D', 10: 'E', 12: 'F'}
res = 1

for z in x:
 if z in y:
 continue
 else:
 res += z

print(res)
```

- a) 31
- b) 28
- c) 30
- d) 29

Answer: d) 29

Topics: tuple dictionary for if else continue

Domain: Control Flow

Explanation:



Which values in the tuple are NOT also indexes in the dictionary?

1 + 7 + 9 + 11 -> 28

The result starts at 1

Makes 29 in total.

---

What is the expected output of the following code?

```
data = ()
print(data.__len__())
```

- a) 1
- b) None
- c) 0
- d) The code is erroneous.

Answer: c) 0.

Topics: `__len__()` tuple

Domain: Data Collections

Explanation:

Every object has the method `__len__()`

Normally we use the function `len()` which calls the method `__len__()`

And sure, the length of an empty tuple is 0

---

What is the expected output of the following code?

```
data = (1,) * 3
data[0] = 2
print(data)
```

- a) (2, 1, 1)
- b) (2, 2, 2)
- c) The code is erroneous.
- d) (1, 1, 1)

Answer: c) The code is erroneous.

Topics: tuple, multiply operator tuple concatenation, tuple indexing

Domain: Data Collections

Explanation:

A tuple is immutable. You can not change it.

Therefore you can not assign something to one of its indexes.

---

What is the expected output of the following code?

```
data1 = 'a', 'b'
data2 = ('a', 'b')
print(data1 == data2)
```

- a) True
- b) 0
- c) False
- d) 1

Answer: a) True

Topics: tuple equal to operator

Domain: Data Collections

Explanation:

You do not need the parentheses to create a tuple

The second tuple will be exactly the same.

And because a tuple is immutable the second tuple will reference to the same object than the first tuple

And sure, their value will be the same, too.

is tests for identity of the object.

== tests for equality of the values.

Try it yourself:

```
data1 = 'a', 'b'
data2 = ('a', 'b')
print(data1 == data2) # True
print(data1 is data2) # True
print(id(data1)) # e.g. 140539383900864
print(id(data2)) # e.g. 140539383900864 (the same number)
```

---

What is the expected output of the following code?

```
data1 = '1', '2'
data2 = ('3', '4')
print(data1 + data2)
a) (1, 2, 3, 4)
b) ['1', '2', '3', '4']
c) ('1', '2', '3', '4')
d) The code is erroneous.
```

Answer: c) ('1', '2', '3', '4')

Topics: plus operator tuple concatenation

Domain: Data Collections

Explanation:

You do not need the parentheses to create a tuple

The rest is normal tuple concatenation.

---

What is the expected output of the following code?

```
data = (1, 2, 4, 8)
data = data[-2:-1]
data = data[-1]
print(data)
a) 4
b) (4,)
c) (4)
d) 44
```

Answer: a) 4

Topics: tuple slicing tuple indexing

Domain: Data Collections

Explanation:

The slicing start is -2 the second last element.

Here is the value 4

The slicing end is -1 the last element, which is exclusive.

That leaves the value 4 in the tuple

The indexing -1 also takes the last element, the value 4  
(which is the only element)

Try it yourself:

```
data = (1, 2, 4, 8)
data = data[-2:-1]
print(data) # (4,)
data = data[-1]
```

```
print(data) # 4
```

---

The fact that tuples belong to sequence types means:

- a) they can be extended using the .append() method
- b) they are actually lists
- c) they can be indexed and sliced like lists
- d) they can be modified using the del instruction

Answer: c) they can be indexed and sliced like lists

Topics: tuples sequences

Domain: Data Collections

---

What is the expected output of the following code?

```
data = (1, 2, 3, 4)
data = data[-2:-1]
data = data[-1]
print(data)
```

- a) 2
- b) 4
- c) 1
- d) 3

Answer: d) 3

Topics: tuple slicing tuple indexing

Domain: Data Collections

Explanation:

Slicing with negative indexes.

A start of -2 takes the second last element.

That would be the value 3

The index -1 is the last element.

The end is always exclusive.

That leaves (3,)

The index -1 is again the last element of that,  
which is the value 3

---

What will be the output after the following code snippet?

```
tup = (1, 2, 4, 8)
tup = tup[1:-1]
tup = tup[1]
print(tup)
```

- 1. 8
- 2. 2
- 3. 4
- 4. 1

Answer: c) 4

Overall explanation

tup[1:-1] means "take all elements starting with the second element until the last element exclusive". This gives (2, 4). Then, tup[1] means "take the second element", which is 4.

---

What will be the output of the following code?

```
tup_a = (0, 1, 2)
```

```
tup_b = (2, 3, 4)
list_c = []
for el in tup_a:
 if el not in tup_b:
 list_c.append(el)
print(list_c)
```

a) [0, 1, 3, 4]  
b) [0, 1]  
c) (0, 1)  
d) (0, 1, 3, 4)

Answer: [0,1]

Overall explanation

This code will essentially fill list\_c with those elements from tup\_a that are NOT present in tup\_b. In other words, we're looking for unique elements in tup\_a. These elements are 0 and 1.

## PCEP-30-02 3.3 Collect and process data using dictionaries

### Topics

- dictionaries: building, indexing, adding and removing keys
- iterating through dictionaries and their keys and values
- checking the existence of keys
- methods: keys(), items(), and values()

### Question and Answers

What code would you insert instead of the comment to obtain the expected output?

Expected output:

a  
b  
c

Code:

```
dictionary = {}
my_list = ['a', 'b', 'c', 'd']

for i in range(len(my_list) - 1):
 dictionary[my_list[i]] = (my_list[i],)

for i in sorted(dictionary.keys()):
 k = dictionary[i]
 # Insert your code here.
```

a) print(k["0"])  
b) print(k)  
c) print(k['0'])  
d) print(k[0])

Answer: d) print(k[0])

Topics: dictionary list for range() tuple dict.keys() sorted()

Domain: Data Collections

Explanation:

You need k[0] because every element of the dictionary is a tuple

and you want the first index of each of those tuples.

```
dictionary = {}
my_list = ['a', 'b', 'c', 'd']

for i in range(len(my_list) - 1):
for i in range(3):
 dictionary[my_list[i]] = (my_list[i],)
print(dictionary) # {'a': ('a',), 'b': ('b',), 'c': ('c',)}

for i in sorted(dictionary.keys()):
for i in dictionary.keys():
 k = dictionary[i]
 # print(k) # ('a',) ('b',) ('c',)
 # print(k[0]) # TypeError: tuple indices must be integers or slices, not str
 # print(k["0"]) # TypeError: tuple indices must be integers or slices, not str
 print(k[0])
```

---

What is the expected output of the following code?

```
data = {'1': '0', '0': '1'}
```

```
for d in data.values():
 print(d, end=' ')
```

- a) 0 0
- b) 1 0
- c) The code is erroneous.
- d) 0 1

Answer: c) The code is erroneous

Topics: dictionary values() for

Domain: Data Collections

Explanation:

This question is a little tricky.

Everything seems to be fine,

BUT the right name of the dictionary method is values()

---

Consider the following code.

```
room = input('Enter the room number: ')
rooms = {101: 'Gathering Place', 102: 'Meeting Room'}
if not room in rooms:
 print('The room doesn\'t exist.')
else:
 print('The room name is: ' + rooms[room])
```

Why is it not working?

- a) None of the above.
- b) Mismatched data type(s)
- c) Misnamed variable(s)
- d) Invalid Syntax

Answer: b) Mismatched data type(s)

Topics: if else dictionary membership operator

Domain: Control Flow

Explanation:

The reason, why the code will never find a room is, that input() always returns a string

and the indexes of the dictionary are all an integer

The issue in the code is due to a mismatch in data types. The `room` variable is obtained from user input, which is a string type, while the keys in the `rooms` dictionary are integers. This causes the `room` variable to not match any keys in the dictionary, resulting in the 'The room doesn't exist.' message being printed.

---

What is the expected output of the following code?

```
data = {'z': 23, 'x': 7, 'y': 42}
```

```
for _ in sorted(data):
 print(data[_], end=' ')
```

- a) 7 42 23
- b) 42 23 7
- c) 7 23 42

Answer: 7 42 23

Topics: dictionary for sorted() print() with end parameter

Domain: Data Collections

Explanation:

The sorted() function returns a sorted

list from the elements in a dictionary (or any other iterable).

In this case the list ['x', 'y', 'z']

In the for loop those indexes get printed out.

---

What is the expected output of the following code?

```
data = {}
data[1] = 1
data['1'] = 2
data[1.0] = 4
```

```
res = 0
for d in data:
 res += data[d]
```

```
print(res)
```

- a) 6
- b) 3
- c) The code is erroneous.
- d) 7

Answer: a) 6

Topics: dictionary indexes for add and assign operator

Domain: Data Collections

Explanation:

Dictionary can have different data types as a key.

BUT if an integer and a float have the same value, the second value overrides the first values.

And the first index is still name giving.

Try it yourself:

```
data = {}
data[1] = 1
data['1'] = 2
data[1.0] = 4

res = 0
for d in data:
 res += data[d]

print(res) # 6

print(data) # {1: 4, '1': 2}
```

---

What is the expected output of the following code?

```
data = {'name': 'Peter', 'age': 30}
person = data.copy()
print(id(data) == id(person))
a) False
b) 1
c) True
d) 0
```

Answer: a) False

Topics: dictionary copy() id() equal to operator

Domain: Data Collections

Explanation:

The copy() method creates a copy of the dictionary

If you just assign the dictionary a reference is created.

---

What is the expected output of the following code?

```
data = {}
data['2'] = [1, 2]
data['1'] = [3, 4]

for i in data.keys():
 print(data[i][1], end=' ')
a) 4 2
b) 1 3
c) 2 4
d) 3 1
```

Answer: c) 2 4

Topics: dictionary for keys() print() with end parameter

Domain: Data Collections

Explanation:

The dictionary does not automatically get sorted.

The key 2 was put in first.

The key 2 will be read first.

The keys() method reads out the keys.

In the first iteration off the for loop i is going to be 2

and in the second iteration i is going to be 1

Try it yourself:

```
data = {}
data['2'] = [1, 2]
data['1'] = [3, 4]

for i in data.keys():
 print(data[i][1], end=' ') # 2 4

print()
print(data) # {'2': [1, 2], '1': [3, 4]}
```

---

What is the expected output of the following code?

```
data = {1: 0, 2: 1, 3: 2, 0: 1}
x = 0
```

```
for _ in range(len(data)):
 x = data[x]
```

```
print(x)
```

- a) 0
- b) The code is erroneous.
- c) 2
- d) 1

Answer: a) 0

Topics: dictionary for range() len()

Domain: Data Collections

Explanation:

The dictionary has four elements  
and therefore the for loop has four iterations.

The value of x gets changed four times: 0 -> 1 -> 0 -> 1 -> 0

And in the end the value again is 0

Try it yourself:

```
data = {1: 0, 2: 1, 3: 2, 0: 1}
x = 0
```

```
for _ in range(len(data)):
 print(x) # 0 - 1 - 0 - 1
 x = data[x]
```

```
print(x) # 0
```

---

What will be the output of the following code?

```
x = [
 'a',
 'b',
 {
 'one': 1,
 'two':
 {
```



```
'x' : 10,
'y' : 20,
'z' : 30
,
'three': 3
,
'c',
'd'
]
print(30 in x[2])
```

- a) False
- b) True
- c) 'z': 30
- d) Python will show an error

Answer: a) False

Overall explanation

This is a very tricky and complex structure that we need to analyse step by step. `x[2]` is the third element in the list. The third element is a complex dictionary between curly brackets { ... }. Now, `30 in x[2]` means: is there a key 30 in the dictionary? The dictionary has no such key (the keys available are: 'one', 'two', 'three'), so the answer is False.

---

What will be the output of the following code snippet?

```
dict1 = {'one': 1, 'two': 2, 'three': 3}
dict2 = {'one': 1, 'two': 5, 'four': 8}
dict3 = dict(dict1)
dict4 = dict(dict2)
dict3.update(dict2)
dict4.update(dict1)
print(dict3 == dict4)
a) The answer depends on the operating system
b) Python will show an error
c) True
d) False
```

Answer: d) False

Overall explanation

`dict3 = dict(dict1)` means "create a new dictionary in variable `dict3` identical to `dict1`". `dict3 = dict(dict2)` works in a similar way. That gives us:

1. `dict3 = {'one': 1, 'two': 2, 'three': 3}`
2. `dict4 = {'one': 1, 'two': 5, 'four': 8}`

`dict3.update(dict2)` means "take all keys from `dict2` and put them alongside their values into `dict3`". This gives us:

1. `dict3 = {'one': 1, 'two': 5, 'three': 3, 'four': 8}`

Note that `dict3` contains key 'three' that was not present in `dict2`. Note also that `dict3` has an updated value for the key 'two'.

Similarly for `dict4.update(dict1)`:

```
dict4 = {'one': 1, 'two': 2, 'four': 8, 'three': 3}
```

Compare `dict3` and `dict4` carefully: the value for the key 'two' is different, so the output is False.

---

## PCEP-30-02 3.4 Operate with strings

### Topics

- constructing strings
- indexing, slicing, immutability
- escaping using the \ character
- quotes and apostrophes inside strings
- multi-line strings
- basic string functions and methods

### Question and Answers

What is the expected output of the following code?

```
x = '\\\\'
```

```
print(len(x))
```

- a) The code is erroneous.
- b) 1
- c) 2
- d) 4

Answer: c) 2

Topics: escaping len()

Domain: Fundamentals

Explanation:

The backslash is the character to escape another character.

If you write '\ ' the backslash escapes

the ending single quote to a normal character.

It takes its syntactical meaning

and the single quote becomes a normal character

and it loses its ability to end the string.

And therefore we get the syntax error.

If you write '\\ ' the one backslash escapes the other

and you end up with a string with one normal backslash.

If you write '\\\\ ' it is kind of the same

and you end up with a string with two normal backslashes.

Try it yourself:

1. # print(len('\ ')) # SyntaxError: ...
2. print(len('\\ ')) # 1
3. # print(len('\\\\ ')) # SyntaxError: ...
4. print(len('\\\\\\ ')) # 2

---

What is the expected output of the following code?

```
x = '\\'
```

```
print(len(x))
```

- a) 3
- b) 1
- c) The code is erroneous.
- d) 2

Answer: c) The code is erroneous

Topics: escaping len()

Domain: Fundamentals

Explanation:

The backslash is the character to escape another character.

If you write '\\' the backslash escapes

the ending single quote to a normal character.

It takes its syntactical meaning and the single quote becomes a normal character and it loses its ability to end the string and therefore we get the syntax error.

If you write '\\\\' the one backslash escapes the other and you end up with a string with one normal backslash.

If you write '\\\\' you again have the same problem than with one single backslash.

The first backslash escapes the second one, but the third backslash escapes the ending single quote, the string does not get closed and you get a syntax error.

---

If you want to build a string that reads:

Peter's sister's name's "Anna"

Which of the following literals would you use?

Choose two.

- a) 'Peter\\'s sister\\'s name\\'s \\'Anna\\''
- b) 'Peter's sister's name's "Anna"'
- c) "Peter's sister's name's "Anna""
- d) "Peter's sister's name's \\'Anna\\''"

Answer: a) and c)

- a) 'Peter\\'s sister\\'s name\\'s \\'Anna\\''
- C) "Peter's sister's name's "Anna""

Topics: quotation marks escaping

Domain: Fundamentals

Explanation:

When you use the same kind of quotes inside of a string and outside to delimit the string you have to escape the ones inside of the string

---

What is the expected output of the following code?

```
data = 'Hello@Peter!!'
print(data.lower())
```

- a) None
- b) hello@Peter!!
- c) hellopeter
- d) hello@peter!!

Answer: d) hello@peter!!

Topics: string lower()

Domain: Data Collections

Explanation:

The string method lower() converts all uppercase characters to lowercase characters.

The not uppercase characters do not get touched.

---

Which of the following sentences is true?

```
str1 = 'Peter'
str2 = str1[:]
```

- a) str1 and str2 are different names of the same string.
- b) str1 is longer than str2
- c) str2 is longer than str1
- d) str1 and str2 are different (but equal) strings.

Answer: a) str1 and str2 are different names of the same string.

Topics: string slicing

Domain: Data Collections

Explanation:

Copying with [:] only works with mutable data types like a list

A string is an immutable data type.

Whether you slice the whole thing or just assign it,  
you always end up with a reference to the same object.

Try it yourself:

```
str1 = 'Peter'
str2 = str1[:]
str2 = str1 # The same thing

print(id(str1)) # e.g. 140539652049216
print(id(str2)) # e.g. 140539652049216 (the same number than str1)
print(str1 is str2) # True
print(str1 == str2) # True
```

---

## Sets

### Question and Answers

What is the expected output of the following code?

```
data = set([1, 2, 2, 3, 3, 3, 4, 4, 4, 4])
print(len(data))
```

- a) 2
- b) 4
- c) 1
- d) 10
- e) 3
- f) 0

Answer: b) 4

Topics: list set() len()

Domain: Data Collections

Explanation: A set can not have duplicates.

## Section 4: Functions and Exceptions (28%)

Objectives covered by the block (8 exam items)

## PCEP-30-02 4.1 – Decompose the code using functions

### Topics

- defining and invoking user-defined functions and generators
- the return keyword, returning results
- the None keyword
- recursion

### Question and Answers

What is the expected output of the following code?

```
def func(x):
 if x % 2 == 0:
 return 1
 else:
 return

print(func(func(2)) + 1)
a) 2
b) None
c) 1
d) The code is erroneous.
```

Answer: d) The code is erroneous.

Topics: def return None if else, modulus operator addition operator

Domain: Functions and Exceptions

Explanation:

The condition `x % 2 == 0` tests for even numbers.

The first function call `func(2)` passes an even number and the return value will be 1

The second function call will therefore pass an odd number `func(1)`

The return with no value will return None

And you can not calculate with None

---

What will be the output of the following code snippet?

```
def func(x):
 if x:
 print('1')
 elif x is False:
 print('2')
 else:
 print('3')

func(None)
a) 1
b) 2
c) 3
d) None
```

Answer: c) 3

Overall explanation

None is neither True nor False in Python, so the else branch is executed.

---

Which of the following statements are true?

(Select two answers)

- a) The None value can be compared with variables
- b) The None value can be assigned to variables
- c) The None value can be used as an argument of arithmetic operators
- d) The None value cannot be used outside functions

Answer: a) The None value can be compared with variables

b) The None value can be assigned to variables

Topic: None

Domain: Fundamentals

Explanation:

It is true, the None value cannot be used as an argument of arithmetic operators.

But the None value can be assigned and compared to variables.

And that can absolutely happen outside of a function.

---

What is the expected output of the following code?

```
def func(n):
 s = "
 for i in range(n):
 s += '*'
 yield s
```

```
for x in func(3):
 print(x, end="")
```

- a) \*\*\*
- b) \*
- c) The code is erroneous.
- d) \*\*\*\*\*

Answer: d) \*\*\*\*\*

Topics: def yield generator function for range(), print() with end parameter

Domain: Functions and Exceptions

Explanation:

The function call func(3) will return a generator object.

In the first iteration of the for loop that generator object will return \*

The second iteration will return \*\*

and the third iteration will pass \*\*\*

That makes six stars in total: \*\*\*\*\*

---

What is the expected output of the following code?

```
def func(n):
 s = '*'
 for i in range(n):
 s += s
 yield s
```

```
for x in func(2):
 print(x, end="")
```

- a) \*\*\*

- b) \*\*\*\*
- c) \*
- d) \*\*

Answer: b) \*\*\*\*

Topics: def yield for range() print with end parameter

Domain: Functions and Exception

Explanation:

The generator function will produce a generator object with one element: '\*\*\*\*'

The for loop will iterate through that string and print every single character.

The snippet would have the same output if yield gets interchanged with return

If you only have one yield keyword, it belongs in a loop.

Try it yourself:

```
def func(n):
```

```
 s = '*'
```

```
 for i in range(n):
```

```
 s += s
```

```
 yield s
```

```
 # return s
```

```
for x in func(2):
```

```
 print(x, end='') # ****
```

```
print()
```

```
print(func(2)) # <generator object func at ...>
```

```
print(list(func(2))) # ['****']
```

```
s = '*'
```

```
s += s # s = s + s -> s = '*' + '*' -> '**'
```

```
s += s # s = s + s -> s = '**' + '**' -> '****'
```

```
print(s) # ****
```

---

Which of the following items are present in the function header?

- a) function name
- b) function name and parameter list
- c) return value
- d) parameter list

Answer: function name and parameter list

Topics: def parameter function header

Domain: Functions and Exceptions

Explanation: In the function header is the keyword def the function name and the parameter list.

---

Which of the literals below is not a valid function name?

- a) 1function
- b) Func\_1\_tion
- c) Function1
- d) \_function1
- e) Function\_1

Answer: a) 1function

Topic: naming functions

Domain: Functions and Exceptions

Explanation:

A function name has to start with a letter or an underscore.

Then numbers, letters and underscores can follow.

---

What is the expected output of the following code?

```
def func(x):
 if x == 0:
 return 0
 return x + func(x - 1)
```

```
print(func(3))
```

- a) 1
- b) 3
- c) 6
- d) The code is erroneous.

Answer: c) 6

Topics: def recursive function if equal to operator

Domain: Functions and Exceptions

Explanation:

func() is a recursive function, because it calls itself.

A recursive function is kind of like a loop.

Here the condition is  $x == 0$

If  $x$  becomes 0 the recursive calls come to an end and 0 gets returned.

Otherwise the function will be called again with an incremented  $x$

That will happen three times here.

And all those function calls will be added to the original value of  $x$  which was 3

Once the 0 is reached and the function is not called again,

the function calls will become their return values and

$3 + 2 + 1 + 0 \rightarrow 6$

---

What will be the output of the following code?

```
def boo(x):
 if x == 1:
 return x
 return x * boo(x-1)
```

```
print(boo(3))
```

- a) Python will show an error
- b) 1
- c) 3
- d) 6

Answer: d) 6

Overall explanation

Note that this is a recursive function. The function calls itself for any input greater than 1.  $\text{boo}(3)$

$= 3 * \text{boo}(2) = 3 * 2 * \text{boo}(1) = 3 * 2 * 1 = 6.$

---

What will be the output of the following code?

```
def myfun(x):
 if x == 0:
 return 1
```



```
return x + myfun(x-1)
```

```
print(myfun(5))
```

- a) 16
- b) 5
- c) 9
- d) 4

Answer: a) 16

Overall explanation

This is a recursive function that calls itself for any argument different than 0. For myfun(5), it returns 5 + myfun(4). For myfun(4), it returns 4 + myfun(3). This stops at myfun(0), which simply returns 1. In the end:

$\text{myfun}(5) = 5 + \text{myfun}(4) = 5 + 4 + \text{myfun}(3) = \dots = 5 + 4 + 3 + 2 + 1 + \text{myfun}(0) = 5 + 4 + 3 + 2 + 1 + 1 = 16.$

---

What is the output of the following snippet?

```
def func(x, y):
 if x == y:
 return x
 else:
 return func(x, y-1)
```

```
print(func(0, 3))
```

- a) 0
- b) 2
- c) 1
- d) The snippet will cause a runtime error.

Answer: a) 0

Topics: functions recursion if else return equal to operator

Domain: Functions and Exceptions

Explanation:

This is a recursive function gone wrong.

If the recursive function call would be added to y

the result would be  $3 + 2 + 1 \rightarrow 6$

But here, in the end the function just returns x which is 0

---

What is the expected behavior of the following snippet?

```
x = 1
```

```
def a(x):
 return 2 * x
```

```
x = 2 + a(x) # Line 8
```

```
print(a(x)) # Line 9
```

It will:

- a) print 4
- b) cause a runtime exception on Line 9
- c) print 8
- d) print 6
- e) cause a runtime exception on Line 8

Answer: c) print 8

Topics: def return multiplication operator addition operator

Domain: Functions and Exceptions

Explanation:

The first call of the function a() will pass 1 to the function.

$2 * 1 \rightarrow 2$

The function will return 2

$2 + 2 \rightarrow 4$

The second call of the function a() will pass 4 to the function.

$2 * 4 \rightarrow 8$

8 will be printed.

Try it yourself:

```
x = 1
```

```
def a(x):
 return 2 * x
```

```
x = 2 + a(x) # Line 8
print(a(x)) # Line 9
8
```

---

What is the expected output of the following code?

```
def func():
 print(x + 1, end=' ')
```

```
x = 1
func()
print(x)
a) 1 1
b) 1 2
c) 2 2
d) 2 1
```

Answer: d) 2 1

Topics: def scope print() with end parameter

Domain: Functions and Exceptions

Explanation:

There is no variable x inside of the function.

Python then will look for a variable x in the outer scope and read from that.

But that will definitely not change the value of the variable x in the outer scope.

---

What will be the output of the following code?

```
def myfunc():
 print(num + 1, end="")
```

```
num = 5
myfunc()
print(num)
a) 56
```

- b) 55
- c) 65
- d) 66

Answer: c) 65

Overall explanation

myfunc() prints num + 1, which is 6, without a space at the end (end="). Then, immediately after the digit 6, we print 5. This gives us 65.

---

What is the expected output of the following code?

```
def fun(n):
 n **= n
 return n
print(fun(3))
```

- a) 3
- b) 27
- c) True
- d) The program will cause an error
- e) 9

Answer: b) 27

Topics: functions return exponentiation/power operator

Domain: Functions and Exceptions

Explanation:

The fun() function is called with the integer 3

3 to the power of 3 is 27

which is then returned and printed.

---

## PCEP-30-02 4.2 – Organize interaction between the function and its environment

### Topics

- parameters vs. arguments
- positional, keyword, and mixed argument passing 4
- default parameter values
- name scopes, name hiding (shadowing), and the global keyword

### Question and Answers

The following snippet:

```
def func(a, b):
 return b ** a
```

```
print(func(b=2, 2))
```

- a) will output None
- b) will output 2
- c) Your answer is incorrect
- d) will output 4
- e) is erroneous

Answer: e) is erroneous

Topics: functions positional parameters keyword parameters, exponentiation/power operator

Domain: Functions and Exceptions

Explanation:

The order of the arguments is wrong.

You cannot have a keyword argument before a positional argument.

---

What is the expected output of the following code?

```
def func(x=2, y=3):
 return x * y
```

```
print(func(y=2, 3))
```

- a) 6
- b) 2
- c) The code is erroneous.
- d) 4

Answer: c) The code is erroneous.

Topics: def return positional/keyword argument

Domain: Functions and Exceptions

Explanation:

The keyword arguments always have to be at the end of the list of arguments.

In other words: You can not have a keyword argument before a positional argument.

---

A function parameter is a kind of variable accessible:

- a) anywhere in the code
- b) only after the function definition's completion
- c) only inside the function

Answer: c) only inside the function

Topic: function, parameter, scope

Domain: Functions and Exceptions

Explanation: Yes, a function parameter is only accessible inside the function.

---

The ABC company is creating a program that allows customers to log the number of miles biked.

The program will send messages based on how many miles the customer logs. You create the following Python code.

???

```
name = input('What is your name? ')
return name
```

???

```
calories = miles * calories_per_mile
return calories
```

```
distance = int(input('How many miles did you bike this week? '))
```

```
burn_rate = 50
```

```
biker = get_name()
```

```
calories_burned = calc_calories(distance, burn_rate)
```

```
print(biker + ', you burned about', calories_burned, 'calories.')
```

What would you insert instead of ??? and ???

- a) `def calc_calories(miles, calories_per_mile):`
- b) `def get_name():`
- c) `def get_name(name):`
- d) `def calc_calories():`
- e) `def get_name(biker):`
- f) `def calc_calories(miles, burn_rate):`

Answer: a) `def calc_calories(miles, calories_per_mile):` and c) `def get_name(name):`

Topics: `def` input() multiplication operator `int()`

Domain: Control Flow

Explanation:

`biker = get_name()` calls `get_name()` without any arguments  
and therefore you can not have any parameters:

`def get_name():`

`calories_burned = calc_calories(distance, burn_rate)`

calls `calc_calories()` with two arguments

and therefore you need two parameters:

`def calc_calories(miles, calories_per_mile):`

---

What is the expected output of the following code?

`x = 42`

```
def func():
 global x
 print('1. x:', x)
 x = 23
 print('2. x:', x)
```

```
func()
print('3. x:', x)
```

- a) 1. x: 42  
2. x: 42  
3. x: 42
- b) None of the above.
- c) 1. x: 42  
2. x: 23  
3. x: 23
- d) 1. x: 42  
2. x: 23  
3. x: 42

Answer: c) 1. x: 42

2. x: 23

3. x: 23

Topics: `def` global scope

Domain: Functions and Exceptions

Explanation:

The variable `x` existed in the outer scope before the function call.

The `global` keyword turns the variable `x` into a global variable.

From now on it exists in the outer scope and in the function scope.

Therefore the change inside the function effect `x` in the outer scope.

---

What is the expected output of the following code?

```
def func(x):
 global y
 y = x * x
 return y
```

```
func(2)
```

```
print(y)
```

- a) The code is erroneous.
- b) 4
- c) None
- d) 2

Answer: b) 4

Topics: def, return, global

Domain: Functions and Exceptions

Explanation:

The global keyword can be used to read

and write a global variable inside of a function.

Once you write global y then y also exists in the outer scope.

You would not need the return here.

It is of no use, because the return value of func(2)

does not get processed any further.

But still you can print y because y is global

---

What is the expected output of the following code?

```
def func(x=2, y=3):
 return x * y
```

```
print(func(y=2))
```

- a) 2
- b) The code is erroneous.
- c) 6
- d) 4

Answer: d) 4

Topics: def default parameter

Domain: Functions and Exceptions

Explanation:

This is an easy question.

It is good to be able to detect that.

There is not always a hidden complexity.

Click the right answer and move on.

Try it yourself:

```
def func(x=2, y=3):
 return x * y
```

```
print(func(y=2)) # 4
```

---

What is the expected output of the following code?

```
people = {}
```

```
def add_person(index):
 if index in people:
 people[index] += 1
 else:
 people[index] = 1
```

```
add_person('Peter')
add_person('Paul')
add_person('peter')
```

```
print(len(people))
```

- a) 1
- b) 2
- c) The code is erroneous.
- d) 3

Answer: d) 3

Topics: def shadowing dictionary

Domain: Functions and Exceptions

Explanation:

Peter is a different index than peter  
because P is a different character than p  
and therefore the dictionary will have three entries.

---

What will be the output of the following snippet?

```
def show_truth():
 mysterious_var = ['New Surprise!']
 print(mysterious_var)
```

```
mysterious_var = ['Surprise!']
print(mysterious_var)
show_truth()
print(mysterious_var)
```

- a) ['Surprise!']  
    ['New Surprise!']  
    ['Surprise!']
- b) ['Surprise!']  
    ['Surprise!']  
    ['Surprise!']
- c) ['Surprise!']  
    ['New Surprise!']  
    ['New Surprise!']
- d) ['New Surprise!']  
    ['New Surprise!']  
    ['New Surprise!']

Answer: a)

['Surprise!']

['New Surprise!']

['Surprise!']

Overall explanation

Due to the concept of shadowing, inside the function we get the new value for the list. However, that value is only present locally within the function. When we exit the function, we can still see the old value.

---

A variable defined outside a function:

- a) may not be accessed in any way inside the function
- b) may be read, but not written (something more is needed to do so)
- c) may be freely accessed inside the function

Answer: b) may be read, but not written (something more is needed to do so)

Topics: scope functions shadowing global

Domain: Functions and Exceptions

Explanation:

In Python variables are shadowing into functions.

You can read them, but not write to them.

If you use the global keyword, you can also write to them

---

What is the output of the following snippet?

```
def any():
 print(var + 1, end="")
```

```
var = 1
any()
print(var)
a) 22
b) 11
c) 12
d) 21
```

Answer: d) 21

Topics: functions scope shadowing

Domain: Functions and Exceptions

Explanation:

The variable var shadows inside the function and is readable there.

Therefore, 2 is printed within the function.

After that the value of the variable var is printed, which is 1

---

## PCEP-30-02 4.3 – Python Built-In Exceptions Hierarchy

### Topics

- BaseException
- Exception
- SystemExit
- KeyboardInterrupt
- abstract exceptions
- ArithmeticError



- LookupError
- IndexError
- KeyError
- TypeError
- ValueError

## Question and Answers

What is the expected output of the following code?

```
try:
 raise Exception
except BaseException:
 print('1')
except Exception:
 print('2')
except:
 print('3')
```

- a) 3
- b) 2
- c) The code is erroneous.
- d) 1

Answer: d) 1

Topics: try except Exception BaseException

Domain: Functions and Exceptions

Explanation:

BaseException is the top-most Exception in Python.

Therefore Exception (like any other exception class)

is a subclass of BaseException

And that is why the except BaseException block is executed.

---

What will be the output after the user enters 0?

```
user_input = int(input('What is your value? '))
print(1/user_input)
```

- a) TypeError raised
- b) number 0 shown to the console
- c) ZeroDivisionError raised
- d) ValueError raised

Answer: c) ZeroDivisionError raised

Overall explanation

You can't divide by 0 so Python raises ZeroDivisionError.

---

What is the expected output of the following code?

```
data = [[0, 1, 2, 3] for i in range(2)]
print(data[2][0])
```

- a) The code is erroneous.
- b) 0
- c) 1
- d) 2

Answer: a) The code is erroneous

Topics: list comprehension list indexing IndexError

Domain: Data Collections

Explanation:

range(2) has two elements: 0 and 1

Therefore the outer list will have two elements.

And data[2] does not exist.

This code will throw IndexError

---

What is the expected output of the following code?

```
data = {'a': 1, 'b': 2, 'c': 3}
```

```
print(data['a', 'b'])
```

- a) [1,2]
- b) {'a':1,'b':2}
- c) (1,2)
- d) The code is erroneous.

Answer: d) The code is erroneous.

Topics: dictionary indexing

Domain: Data Collections

Explanation:

You can have a tuple as key of a dictionary

but that is not given here.

print(data['a', 'b']) is incorrect and throws KeyError

print(data['a']) is correct

---

What is the expected output of the following code?

```
num = '7' * '7'
```

```
print(num)
```

- a) The code is erroneous.
- b) 49
- c) Your answer is incorrect
- d) 77
- e) 7777777

Answer: a) The code is erroneous

Topic: TypeError multiplication operator

Domain: Functions and Exceptions

Explanation:

As the error message above states,  
you can only multiply a string by an integer.

Try it yourself:

```
num = '7' * '7' # TypeError: ...
```

```
print('7' * 4) # 7777
```

---

What is the expected behavior of the following program if the user enters 0?

```
value = input("Enter a value: ")
```

```
print(10/value)
```

- a) The program will raise the TypeError exception
- b) The program will raise the ValueError exception
- c) The program will output 0 to the console
- d) The program will raise the ZeroDivisionError exception

Answer: a) The program will raise the TypeError exception

Topics: TypeError input()

Domain: Functions and Exceptions

Explanation:

The input() function always returns a string.

If you try to divide by a string you get a TypeError

---

The following snippet:

```
def function_1(a):
 return None

def function_2(a):
 return function_1(a) * function_1(a)

print(function_2(2))
```

- a) will output 16
- b) will output 4
- c) will output 2
- d) will cause a runtime error

Answer: d) will cause a runtime error

Topics: functions return None multiplication operator TypeError

Domain: Functions and Exceptions

Explanation:

function\_1() returns the None value

and any arithmetic operation with the None value will cause a TypeError.

---

The following snippet:

```
def func(a, b):
 return a ** a

print(func(2))
```

- a) will output 2
- b) will return None
- c) is erroneous
- d) will output 4

Answer: c) is erroneous

Topics: functions parameter exponentiation/power operator

Domain: Functions and Exceptions

Explanation:

There are two parameters without any default values and only one argument.

That causes a TypeError

---

What is the output of the following code?

```
try:
 value = input("Enter a value: ")
 print(value/value)
except ValueError:
 print("Bad input...")
except ZeroDivisionError:
 print("Very bad input...")
```

```
except TypeError:
 print("Very very bad input...")
except:
 print("Booo!")
```

- a) Very bad input...
- b) Very very bad input...
- c) Booo!
- d) Bad input...

Answer: b) Very very bad input...

Topics: try except input() ValueError, TypeError ZeroDivisionError

Domain: Functions and Exceptions

Explanation:

The input() function always returns a string.

If you try to divide by a string you get a TypeError

---

What is the output of the following code if the user enters a 0?

```
try:
 value = input("Enter a value: ")
 print(int(value)/int(value))
except ValueError:
 print("Bad input...")
except ZeroDivisionError:
 print("Very bad input...")
except TypeError:
 print("Very very bad input...")
except:
 print("Booo!")
```

- a) Booo!
- b) 0.0
- c) Very bad input...
- d) Very very bad input...
- e) 1.0
- f) Bad input...

Answer: c) Very bad input...

Topics: try except input() int() division operator, TypeError ValueError ZeroDivisionError

Domain: Functions and Exceptions

Explanation:

input() always returns a string.

After the input the value variable will be "0"

The int() function will cast that to the integer 0

0 / 0 will throw ZeroDivisionError

because you are not allowed to divide by zero.

---

What will be the output of the following program when the user enters the letter a?

```
try:
 val = int(input('Provide a number: '))
 print(val/val)
except TypeError:
 print('a')
```

```
except ValueError:
 print('b')
except ZeroDivisionError:
 print('c')
except:
 print('d')
a) c
b) b
c) a
d) d
```

Answer: b) b

Overall explanation

We're trying to use `int('a')`, which is impossible because 'a' is not a number. This gives us a `ValueError`.

---

What is the expected behavior of the following program?

```
foo = (1, 2, 3)
foo.index(0)
```

- a) The program will cause a `AttributeError` exception.
- b) The program will cause a `TypeError` exception.
- c) The program will output 1 to the screen.
- d) The program will cause a `ValueError` exception.
- e) The program will cause a `SyntaxError` exception.

Answer: d) The program will cause a `ValueError` exception.

Topics: tuple, `tuple.index()`, `ValueError`

Domain: Functions and Exceptions

Explanation

The `index()` method in Python returns the index of the first occurrence of a value in a tuple. If the value is not found in the tuple, it raises a `ValueError` exception, which is the expected behavior in this case.

The `tuple.index()` method expects a value and looks for that value in the tuple.

It will return the index of the first value it finds.

If it does not find the value, it raises a `ValueError`

---

The following statement ...

```
assert x == 0
```

- a) has no effect.
- b) is erroneous.
- c) will stop the program if x is equal to 0
- d) will stop the program if x is not equal to 0

Answer: d) will stop the program if x is not equal to 0

Topic: `assert` `AssertionError`

Explanation:

If the assertion (here `x == 0`) is `True` the program will continue.

Otherwise an `AssertionError` occurs.

Domain: Functions and Exceptions

---

An assertion can be used to:

- a) Stop the program when some data have improper values.
- b) Import a module.
- c) Make the Programmer more assertive.

Answer: a) Stop the program when some data have improper values.

Topic: assertion assert

Domain: Functions and Exceptions

Explanation:

The function `remove_min()` removes the minimal value from a list.

First the function checks, whether the argument is a list

and second the function checks, whether that list has elements.

---

## PCEP-30-02 4.4 – Basics of Python Exception Handling

### Topics

- try-except / the try-except Exception
- ordering the except branches
- propagating exceptions through function boundaries
- delegating responsibility for handling exceptions

### Question and Answers

What is the expected result of the following code?

```
try:
 raise Exception
except:
 print("c")
except BaseException:
 print("a")
except Exception:
 print("b")
```

- a) The code will cause a syntax error.
- b) b
- c) a
- d) 1

Answer: a) The code will cause a syntax error

Topics: try except Exception BaseException SyntaxError

Domain: Functions and Exceptions

Explanation:

As the error message claims:

default 'except:' must be last

---

Which of the following snippets shows the correct way of handling multiple exceptions in a single except clause?

- a) `except TypeError, ValueError, ZeroDivisionError`  
    # Some code.
- b) `except (TypeError, ValueError, ZeroDivisionError)`

- ```
# Some code.  
c) except: (TypeError, ValueError, ZeroDivisionError)  
    # Some code.  
d) except TypeError, ValueError, ZeroDivisionError:  
    # Some code.  
e) except: TypeError, ValueError, ZeroDivisionError  
    # Some code.  
f) except (TypeError, ValueError, ZeroDivisionError):  
    # Some code.
```

Answer: f) except (TypeError, ValueError, ZeroDivisionError):
 # Some code.

Topics: except multiple exceptions

Domain: Functions and Exceptions

Explanation:

In Python the colon always has to be at the end of the line.

The list of possible exceptions has to be in parentheses.

Select the true statements about the try-except block in relation to the following example.

- ```
1. try:
2. # Some code is here...
3. except:
4. # Some code is here...
```

(Select three answers.)

- a) If there is a syntax error in code located in the try block, the except branch will not handle it, and a `SyntaxError` exception will be raised instead.
- b) If you suspect that a snippet may raise an exception, you should place it in the try block.
- c) The code that follows the try statement will be executed if the code in the except clause runs into an error.
- d) The code that follows the except statement will be executed if the code in the try clause runs into an error.

Answer: a), b) and d)

Topic: try except

Domain: Functions and Exceptions

Explanation:

The exception handling will not catch a syntax error.

A syntax error will directly end the script.

---

The unnamed except block ...

- a) can be placed anywhere.
- b) must be the first one.
- c) cannot be used if any named block has been used.
- d) Your answer is correct
- e) must be the last one.

Answer: e) must be the last one

Topic: except `SyntaxError`

Domain: Functions and Exceptions

Explanation:

The unnamed except block always needs to be the last one of the except blocks, otherwise you will receive a SyntaxError

---

What is the output of the following program if the user enters kangaroo at the first prompt and 0 at the second prompt?

```
try:
 first_prompt = input("Enter the first value: ")
 a = len(first_prompt)
 second_prompt = input("Enter the second value: ")
 b = len(second_prompt) * 2
 print(a/b)
except ZeroDivisionError:
 print("Do not divide by zero!")
except ValueError:
 print("Wrong value.")
except:
 print("Error.Error.Error.")
```

- a) 4.0
- b) Error.Error.Error.
- c) Wrong value.
- d) Do not divide by zero!

Answer: a) 4.0

Topics: try except ZeroDivisionError ValueError input(), len() multiplication operator

Domain: Functions and Exceptions

Explanation:

No exception will be thrown.

The length of kangaroo is 8

and the length of 0 is 1

$1 * 2 \rightarrow 2$

$8 / 2 \rightarrow 4.0$

---

The part of your code where the handling of an exception takes place should be placed inside:

- a) the try: branch
- b) the exception: branch
- c) the except: branch

Answer: c) the except: branch

Topic: except

Domain: Functions and Exceptions

Explanation:

It is about exceptions, but the branch is called except:

---

The part of your code where you think an exception may occur should be placed inside:

- a) the exception: branch
- b) the try: branch
- c) the except: branch

Answer: b) the try: branch

Topics: try except

Domain: Functions and Exceptions

---



